COPYRIGHT WARNING: Copyright in these original lectures is owned by Monash University. You may transcribe, take notes, download or stream lectures for the purpose of your research and study only. If used for any other purpose, (excluding exceptions in the Copyright Act 1969 (Cth)) the University may take legal action for infringement of copyright.

Do not share, redistribute, or upload the lecture to a third party without a written permission!

FIT3181/5215 Deep Learning

Week 02: Prelude to Deep Learning and Feed-forward Neural Networks

CE/Lecturer: Dr Trung Le (Clayton)

Lecturer: Dr Binh Nguyen (Clayton)

Lecturer: Dr Ting Fung Fung (Malaysia)

Lecturer: Dr Arghya Pal (Malaysia)

Lecturer: Dr Sailaja Rajanala (Malaysia)

Email: trunglm@monash.edu

GROUP OF EIGHT AUSTRALIA

Department of Data Science and AI
Faculty of Information Technology, Monash University, Australia



Outline

- Prelude to Deep Learning
- Multi-layered feed-forward neural networks (NNs)
  - Evolution of feed-forward NNs
  - Parameterization of feed-forward NNs
  - Modern activation function
  - Forward propagation
- Computational graph of feed-forward neural networks
- Train feed-forward NNs

- **Further reading recommendations**
  - Dive into Deep Learning – Chapter 4: https://d2l.ai/chapter_multilayer-perceptrons/index.html
  - Deep Learning - Chapter 6
  - Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow – Chapter 10

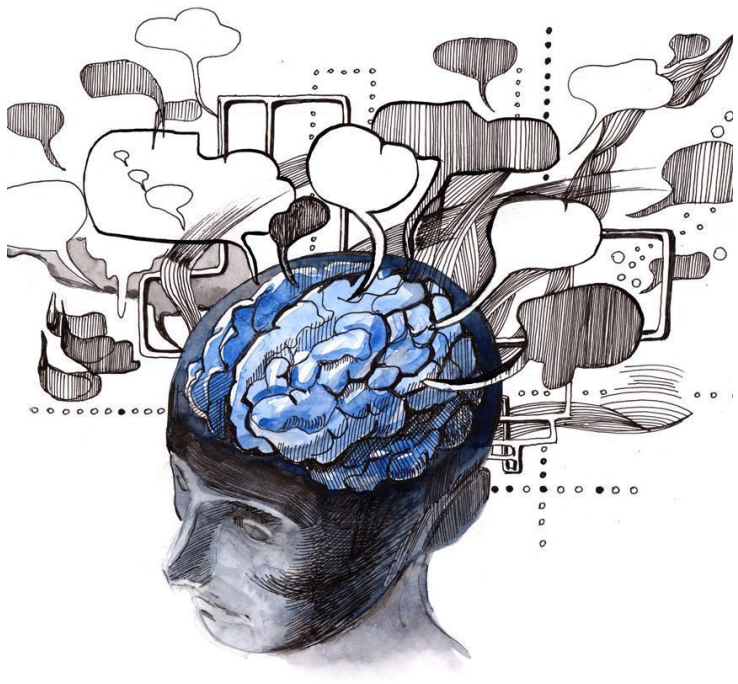
Applications of Deep Learning

MONASH University

Prelude to Deep Learning

A Prelude to Deep Learning

- A young and fast moving field, but with history dated back to 40s (connectionism)!
- Rooted in machine learning and early pattern recognition theory.
- Drawn heavily on knowledge of human brain, statistics and applied maths
- Central technology to build Artificial Intelligence systems.



[Image courtesy: @DeepLearningIT]

A prelude to deep learning

1990 — Dark age of DL — 2012

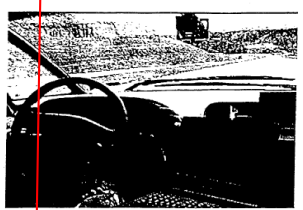
4 nodes, 3 layers

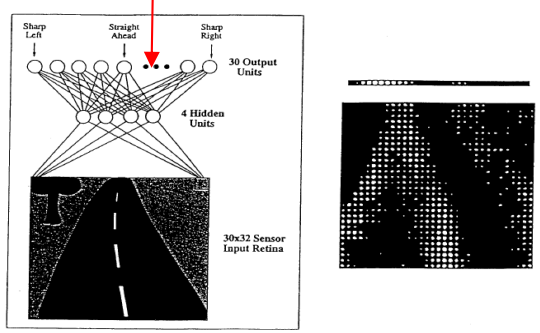
.. and TODAY, 152 – 1000 layers (Resnet)

GPT-5.6: (estimated) 2.2 trillion params

Advancements in hardware: GPU, TPU

AlexNet won ImageNet 2012





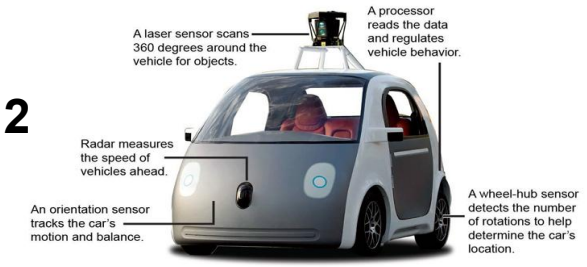
[*Machine Learning*, Tom Mitchell, 1997]

Early day of NN application

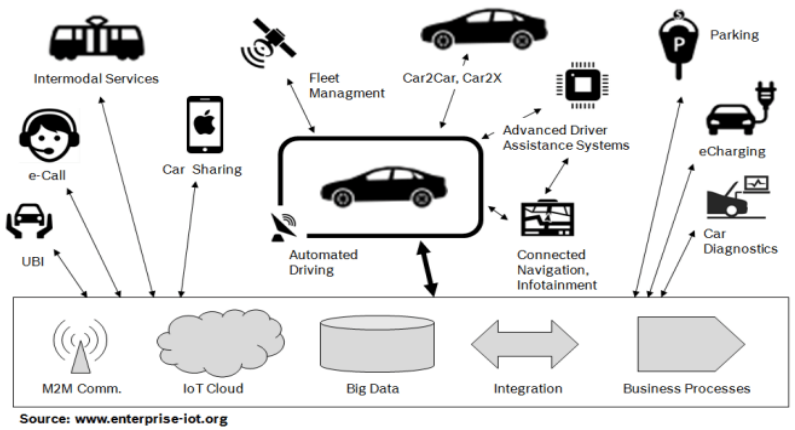
Massive datasets: ImageNet, MSCoco

Better techniques to train DL models:

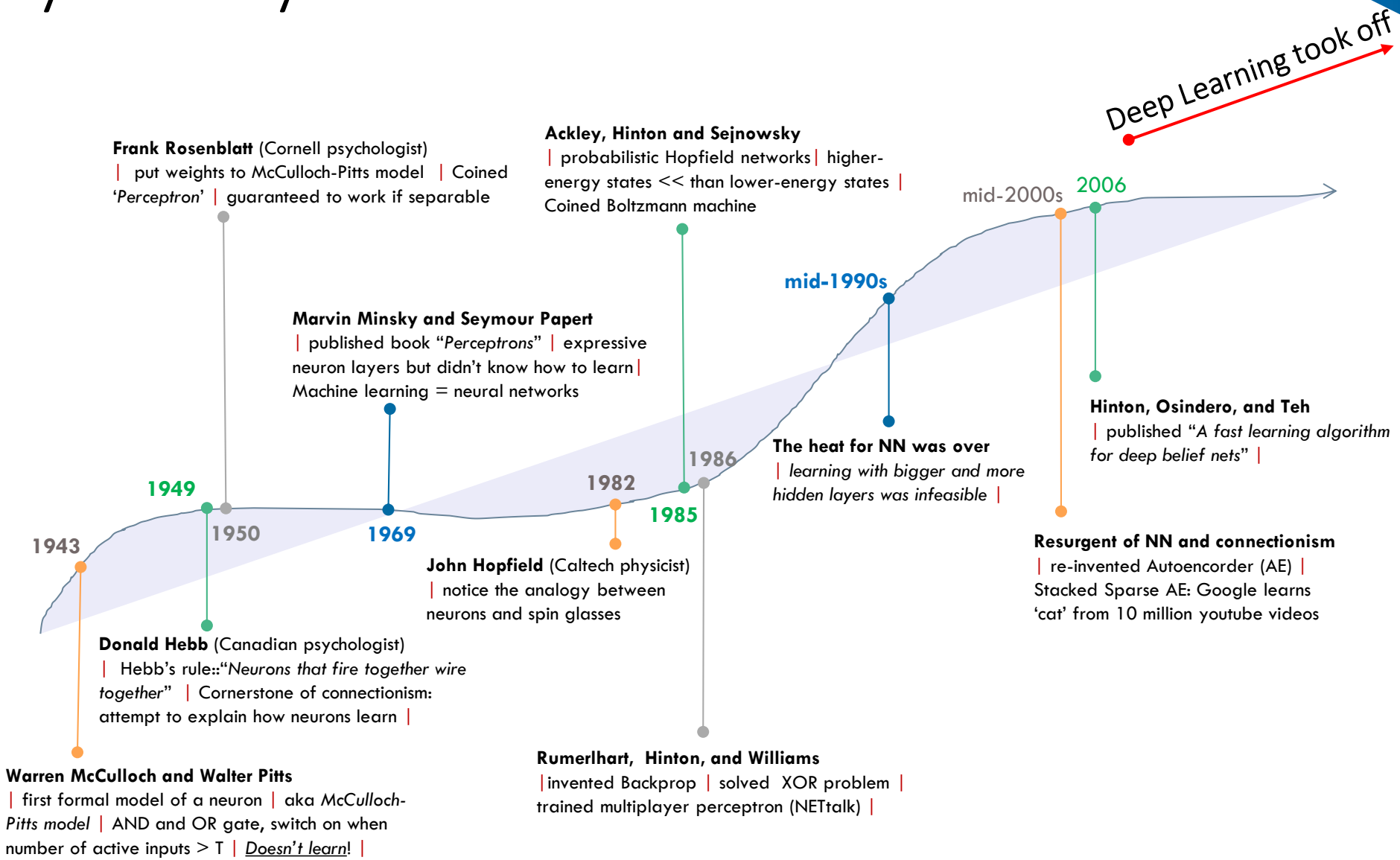
- Adam, RMSProp
- ResNet, DenseNet
- BatchNorm, DropOut, MixUp



Truck, Train, Boat, Uber, UAV, ……



Early history of DL







Yann LeCun
Director of AI Research, Facebook



Yoshua Bengio
Head, Montreal Institute for Learning Algorithms



Geoff Hinton
Google Brain Toronto, University of Torronto

The ACM A.M. Turing Award is an annual prize given by the Association for Computing Machinery (ACM) to "an individual selected for contributions of a technical nature made to the computing community". The Turing Award is recognized as the "highest distinction in computer science" and "Nobel Prize of Computing". The award is named after Alan Mathison Turing, mathematician and reader in mathematics at the University of Manchester. From 2007 to 2013, the award was accompanied by a prize of $250,000. Effective 2014, the funding level has been increased to $1 million, i.e. four times the previous amount.

Deep learning

Yann LeCun[1,2], Yoshua Bengio[3] & Geoffrey Hinton[4,5]

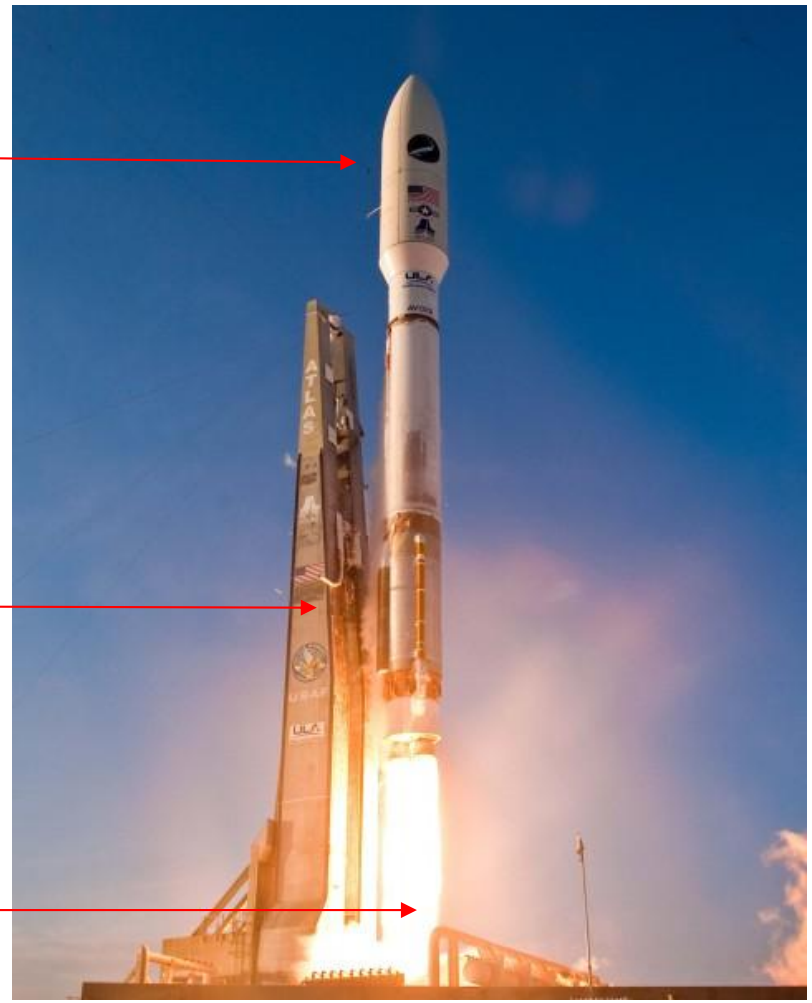
nature

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers intricate structure in large data sets by using the backpropagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representation in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and audio, whereas recurrent nets have shone light on sequential data such as text and speech.

When DL works?



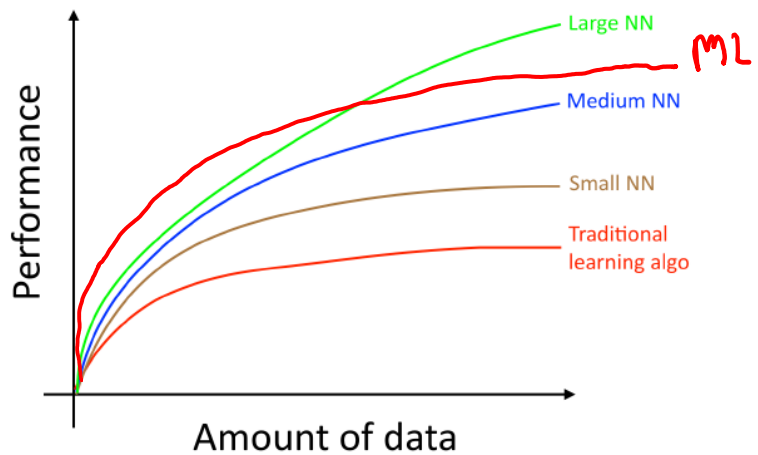
deep learning models





Computational infrastructure, researcher, environment and policy as the launch pad

Big data as energy resource

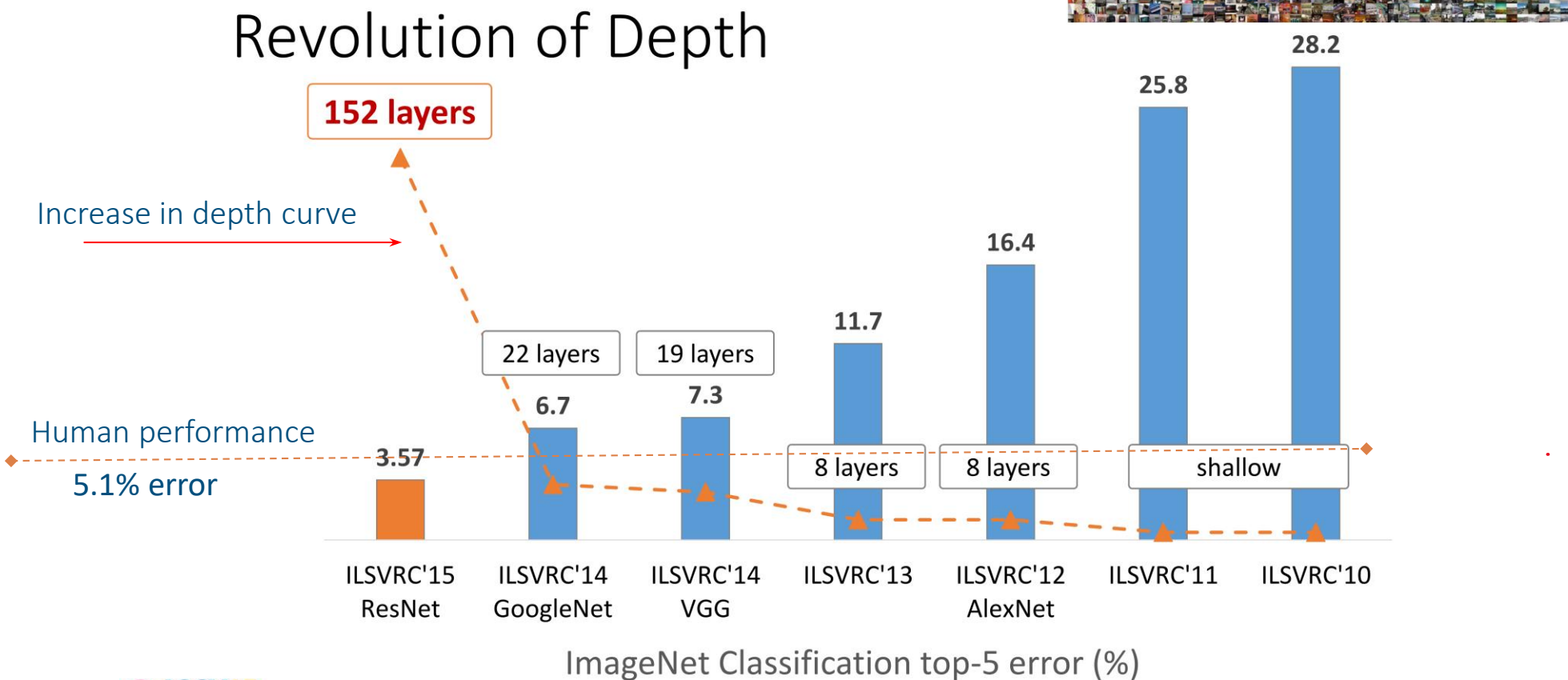


Andrew Ng's Analogy

©Copyright Dinh Phung, 2019

Breakthrough in object recognition





ICCV15 International Conference on Computer Vision

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015.

ResNet 2015 beats human performance in object recognition task!

MONASH University

DL applications

Computer vision/NLP/Speech Analysis and big data

Large Language Models & Generative AI

Speech Recognition & Computer Vision

Drug discovery & Medical Image Analysis

Recommender System



OpenAI Google IBM MERCK amazon.com

Microsoft NOVARTIS ebay NETFLIX

DL applications

Reinforcement learning

Mastering the game of Go with deep neural networks and tree search

David Silver[1]*, Aja Huang[1]*, Chris J. Maddison[1], Arthur Guez[1], Laurent Sifre[1], George van den Driessche[1], Julian Schrittwieser[1], Ioannis Antonoglou[1], Veda Panneershelvam[1], Marc Lanctot[1], Sander Dieleman[1], Dominik Grewe[1], John Nham[2], Nal Kalchbrenner[1], Ilya Sutskever[2], Timothy Lillicrap[1], Madeleine Leach[1], Koray Kavukcuoglu[1], Thore Graepel[1] & Demis Hassabis[1]

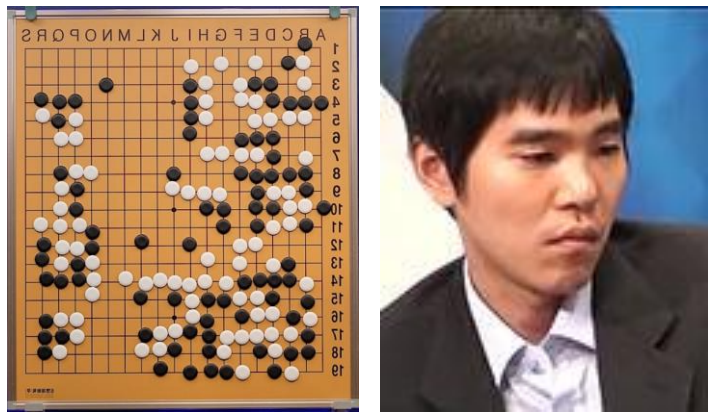


DL applications

Reinforcement Learning



1997



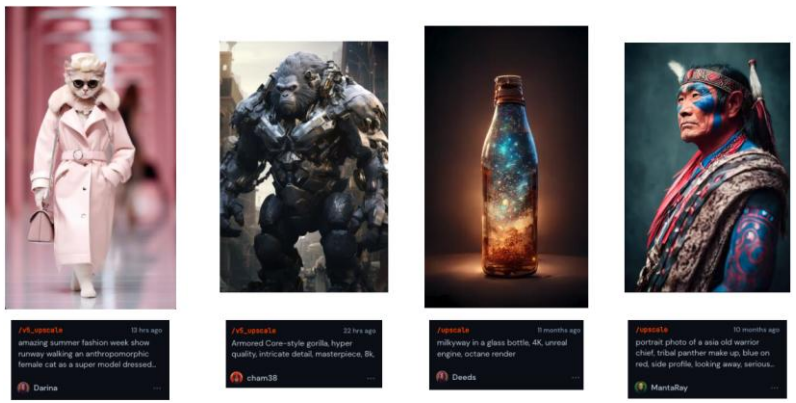
2016

from search heuristics to ...

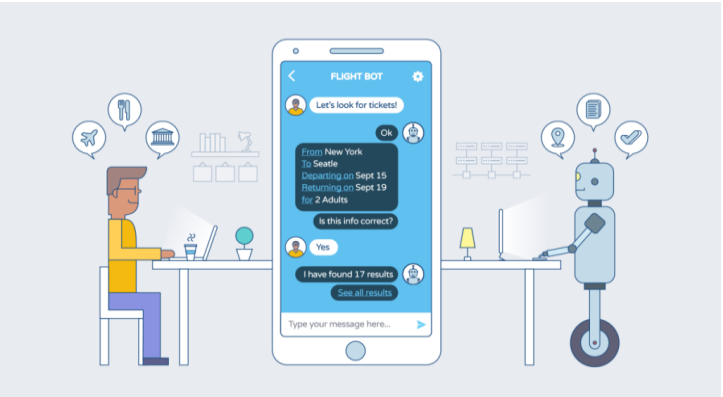
AUTOMATED LEARNING!

Other applications

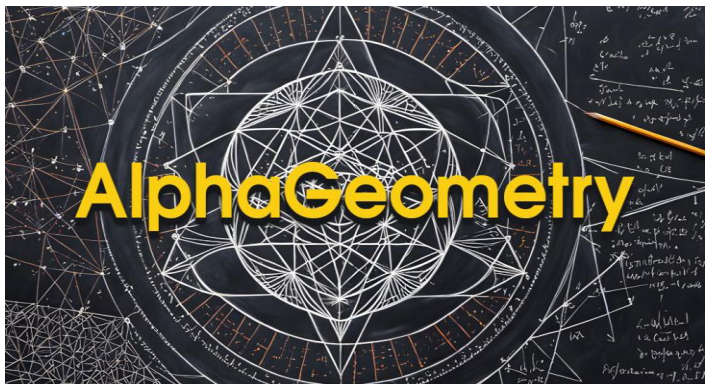
Generative AI
DALL.E/Mid Journey



Chatbot/LLMs







Video Surveillance and Analysis



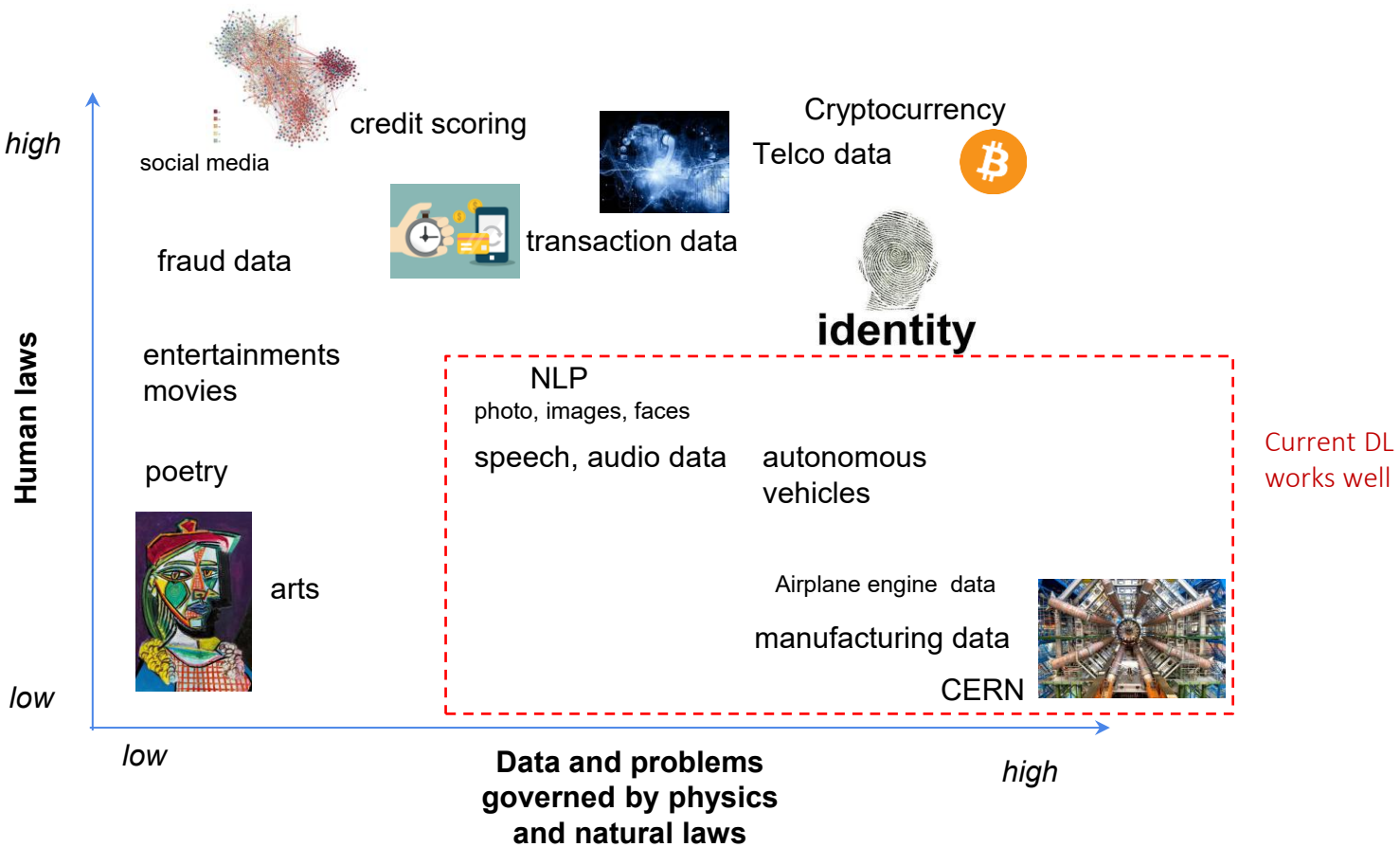
Tesla self-driving cars



Machine translation with seq2seq



Where DL works and doesn't work?



Where does DL sit within ML field?

Approaches	Prob
Symbolists	Kno
Evolutionary	S
Connectionists	
Bayesian	
Analogizers	Similar

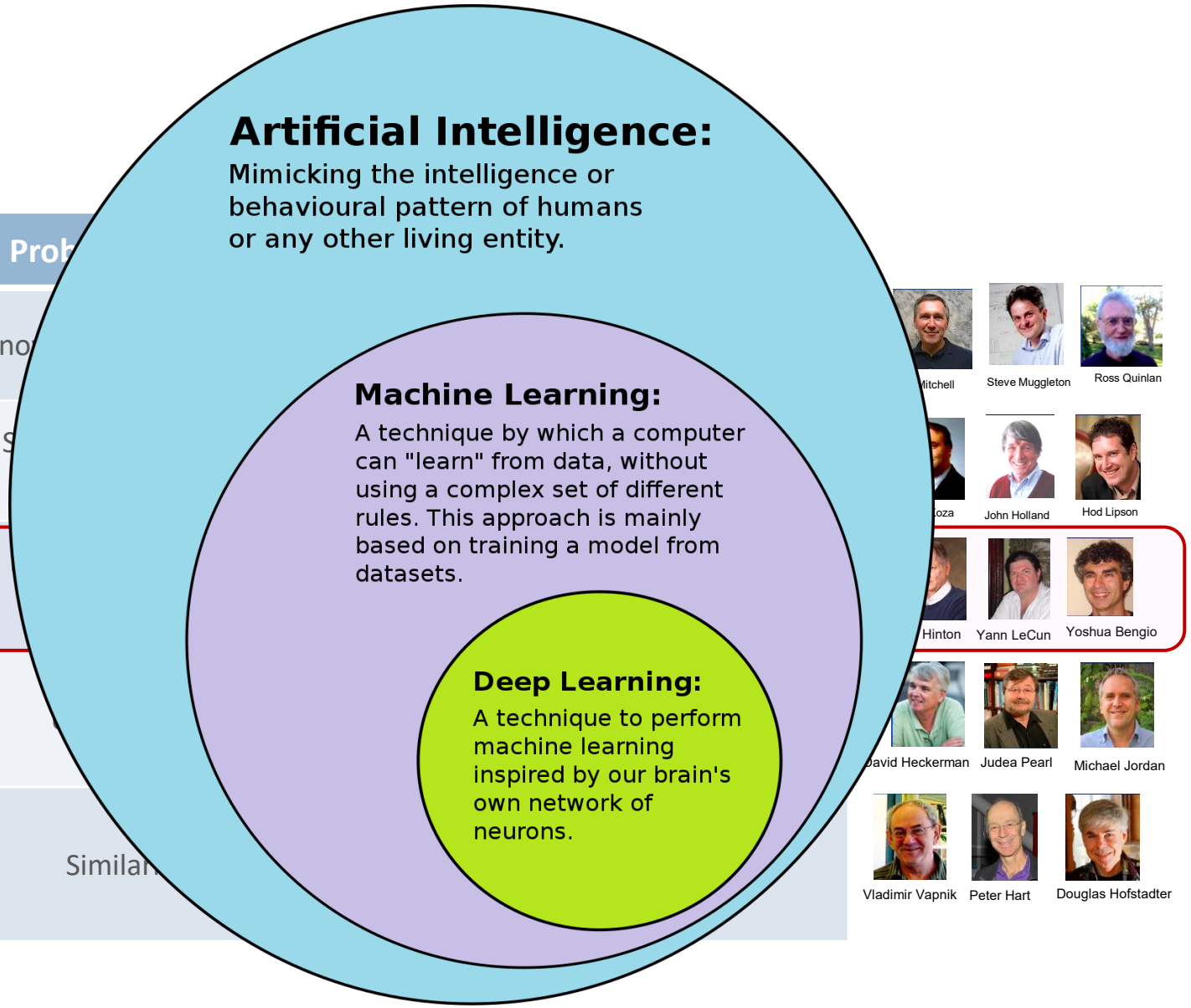


Image credit: Wikipedia

Source: “The Master Algorithm”, Domingo


Q&A
WHO?
WHERE?
WHEN?
WHAT?
How?
Why?
When?
What?
HOW?

Deep Learning Models in This Unit

Discriminative machine learning



Data point x

Ground-truth label

$y = 8 \in \{1, .., 10\}$

feed

ML model h

$h(x)$ | $h_1(x)$, $h_2(x)$, ..., $h_{10}(x)$

softmax

Prediction probabilities

$p(x) = \text{softmax}(h(x))$

$p(x)$ | $p_1(x)$, $p_2(x)$, ..., $p_{10}(x)$

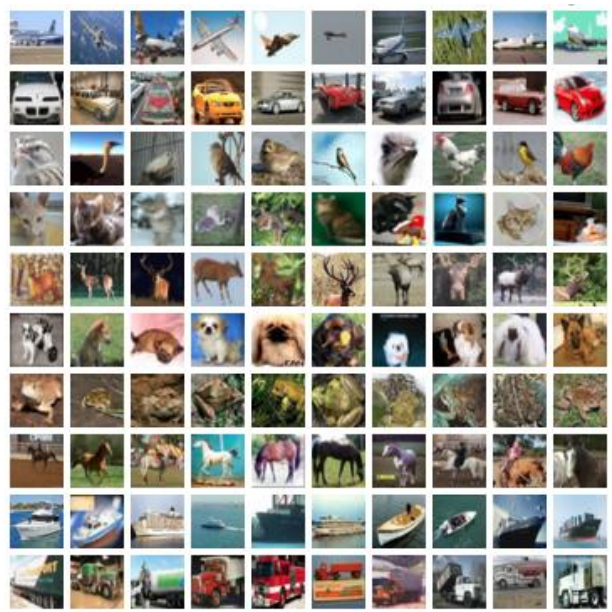
predict

$f(x) = \hat{y} := \text{argmax}_{1 \le m \le 10} p_m(x)$

Suffer **loss** $l(y, \hat{y}) = CE\left(1_y, p(x)\right) = -\log p_y$

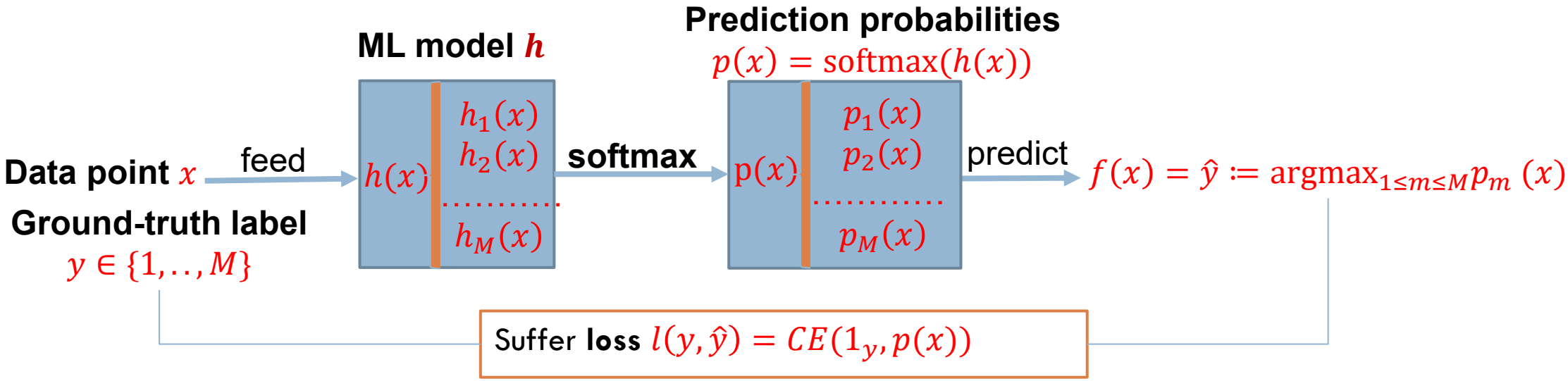
- $h_1 = -1, h_2 = h_3 = h_4 = h_5 = h_6 = 1, h_7 = 5, h_8 = 3, h_9 = h_{10} = 0$
  - Possibilities to assign data example x to classes.
- $p_1 = 0.002, p_2 = p_3 = p_4 = p_5 = p_6 = 0.015, p_7 = 0.8, p_8 = 0.11, p_9 = p_{10} = 0.005$
  - $\hat{y} = 7 \neq y = 8$ → incorrect prediction.
  - $l(y, \hat{y}) = \text{CE}(1_y, p) = -\log p_y = -\log p_8 = -\log 0.11$

(1) airplane
(2) automobile
(3) bird
(4) cat
(5) deer
(6) dog
(7) frog
(8) horse
(9) ship
(10) truck

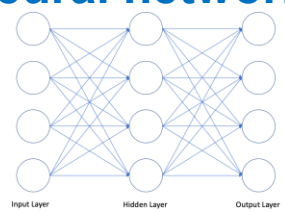


CIFAR 10 dataset

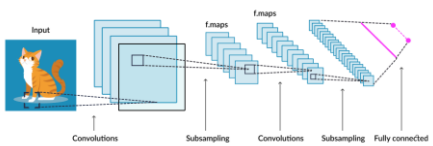
Deep learning



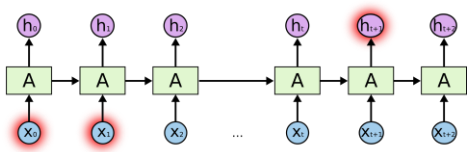
Deep neural networks with multiple neural layers and neurons



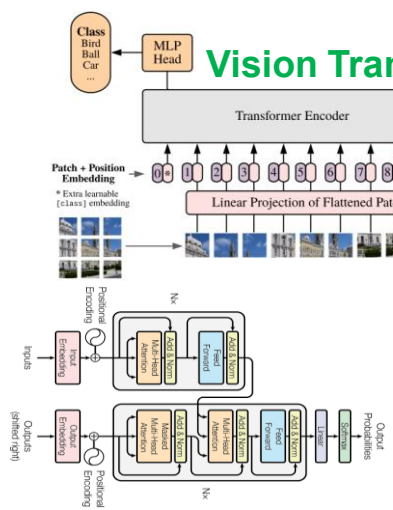
Multilayered feedforward nets



Convolutional neural nets



Recurrent neural nets

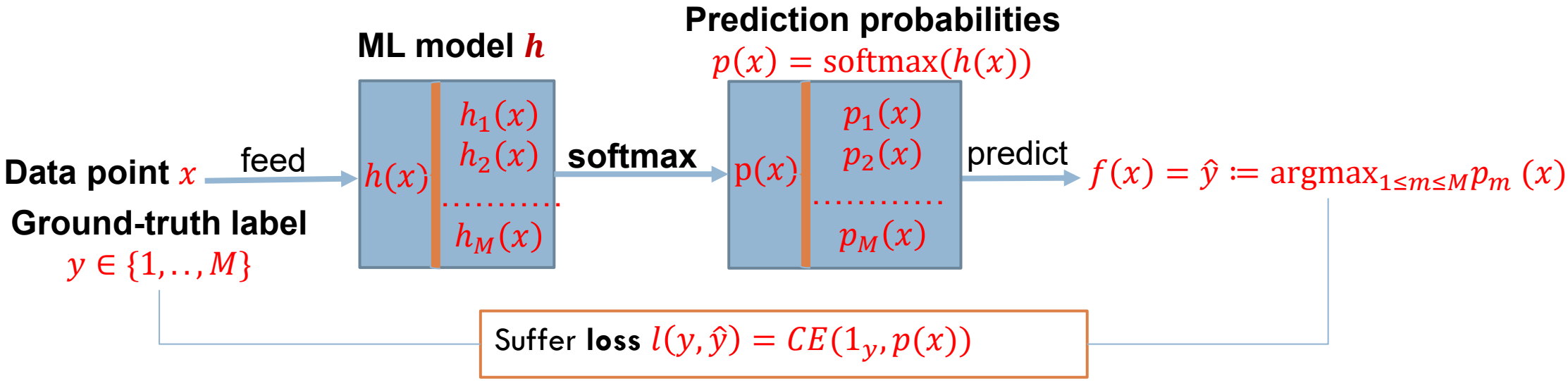


Vision Transformers

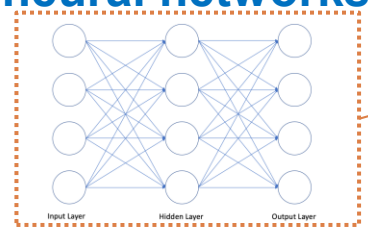
Transformers

- **Several SOTA deep nets** will be **covered** in this unit:
  - Multilayered feedforward neural nets
    - For working with **vectors**, **1D tensors**
  - Convolutional neural nets/ViT
    - For working with **images**, **2D/3D tensors**
  - Recurrent neural nets/Transformers
    - For working with **sequences**, **sentences, texts**

Deep learning

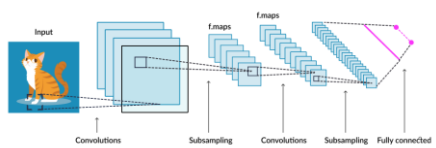


Deep neural networks with multiple neural layers and neurons

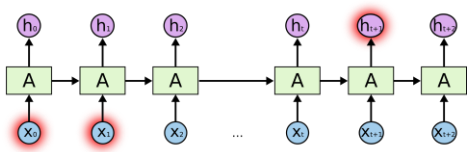


Our **main character** today

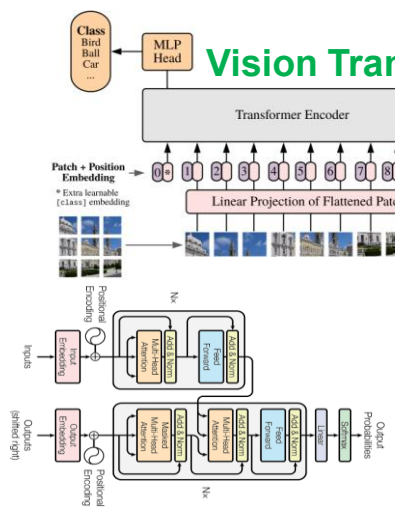
Multilayered feedforward nets



Convolutional neural nets



Recurrent neural nets



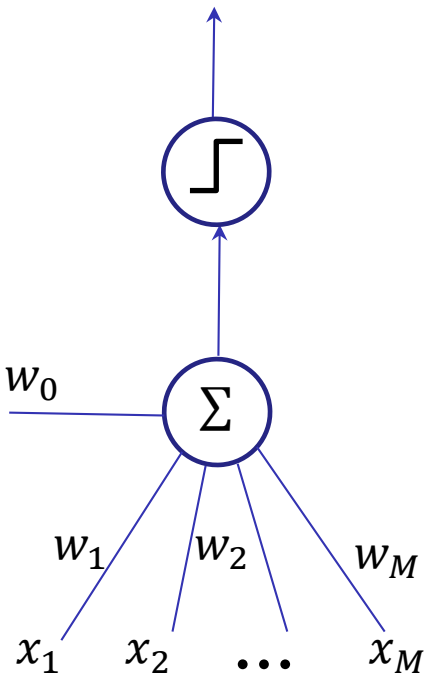
Vision Transformers

Transformers

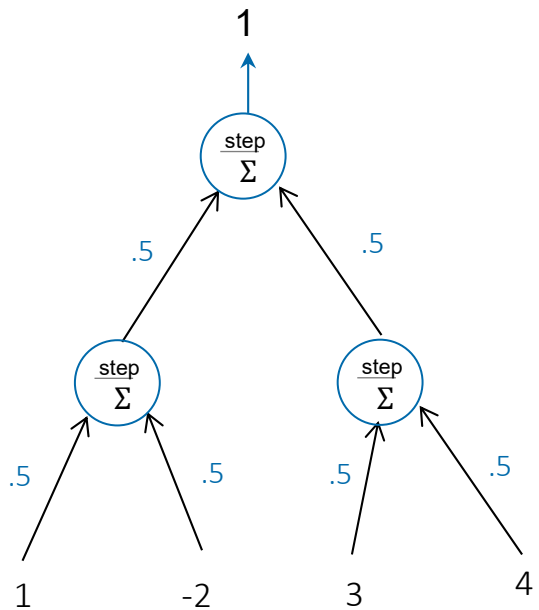
- **Several SOTA deep nets** will be **covered** in this unit:
  - Multilayered feedforward neural nets
    - For working with **vectors**, **1D tensors**
  - Convolutional neural nets/ViT
    - For working with **images**, **2D/3D tensors**
  - Recurrent neural nets/Transformers
    - For working with **sequences**, **sentences, texts**

Feed-forward Neural Networks

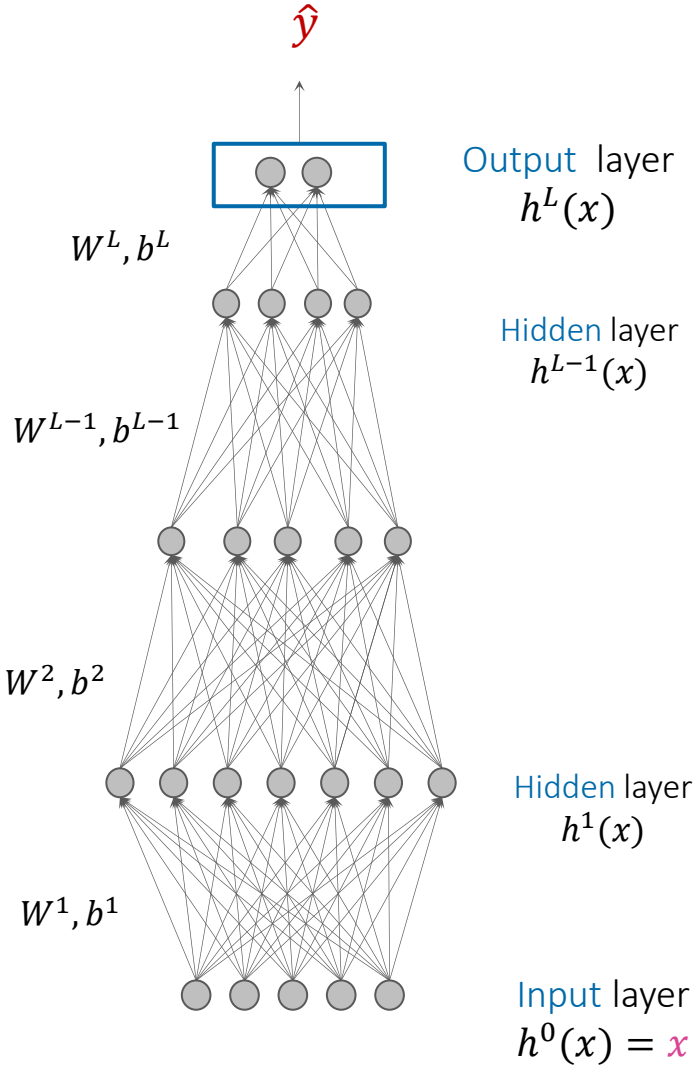
Evolution of Feed-forward Neural Networks



Single Perceptron

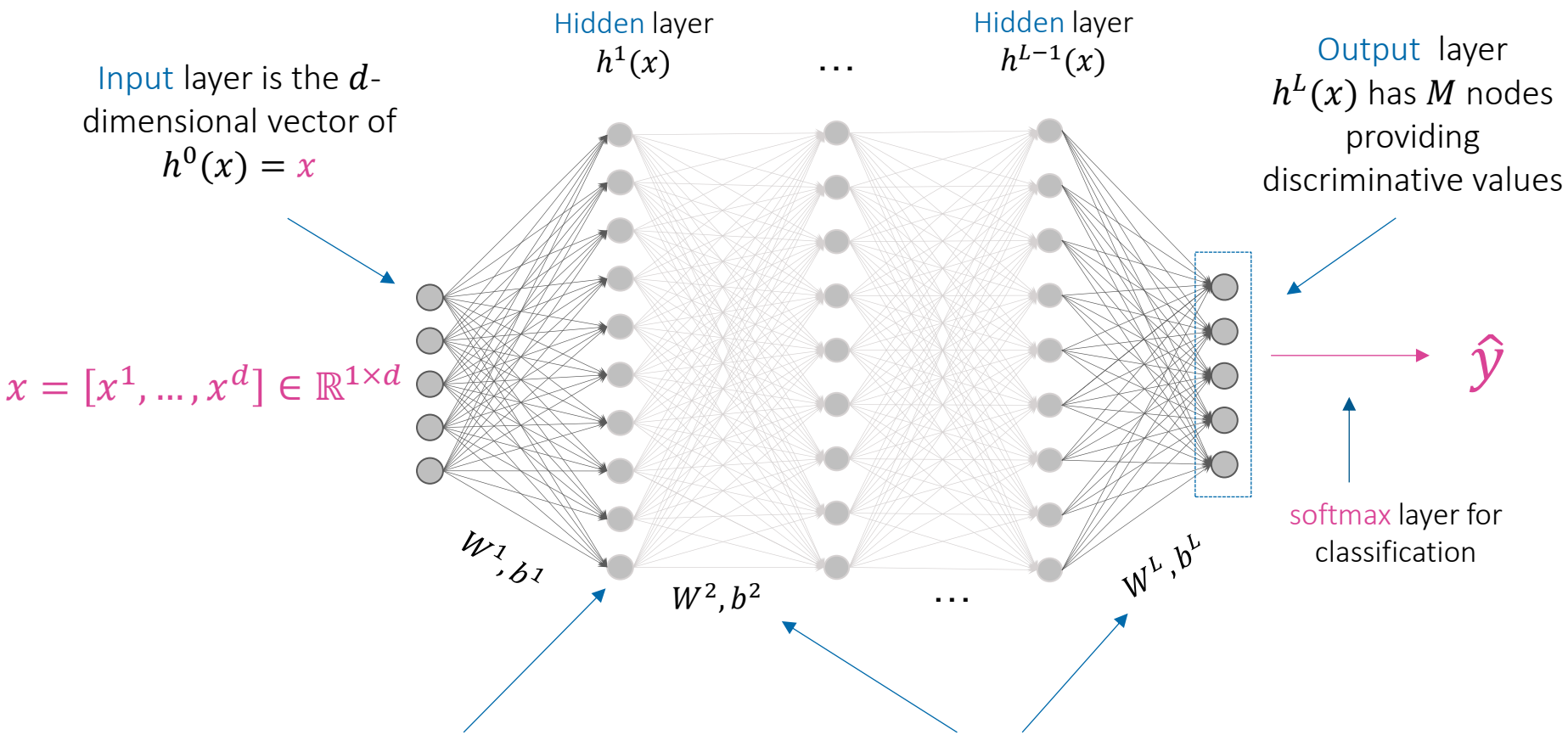


Multi-layered Perceptron



Multi-layered Feedforward NN

Deep NNs: parameterisation



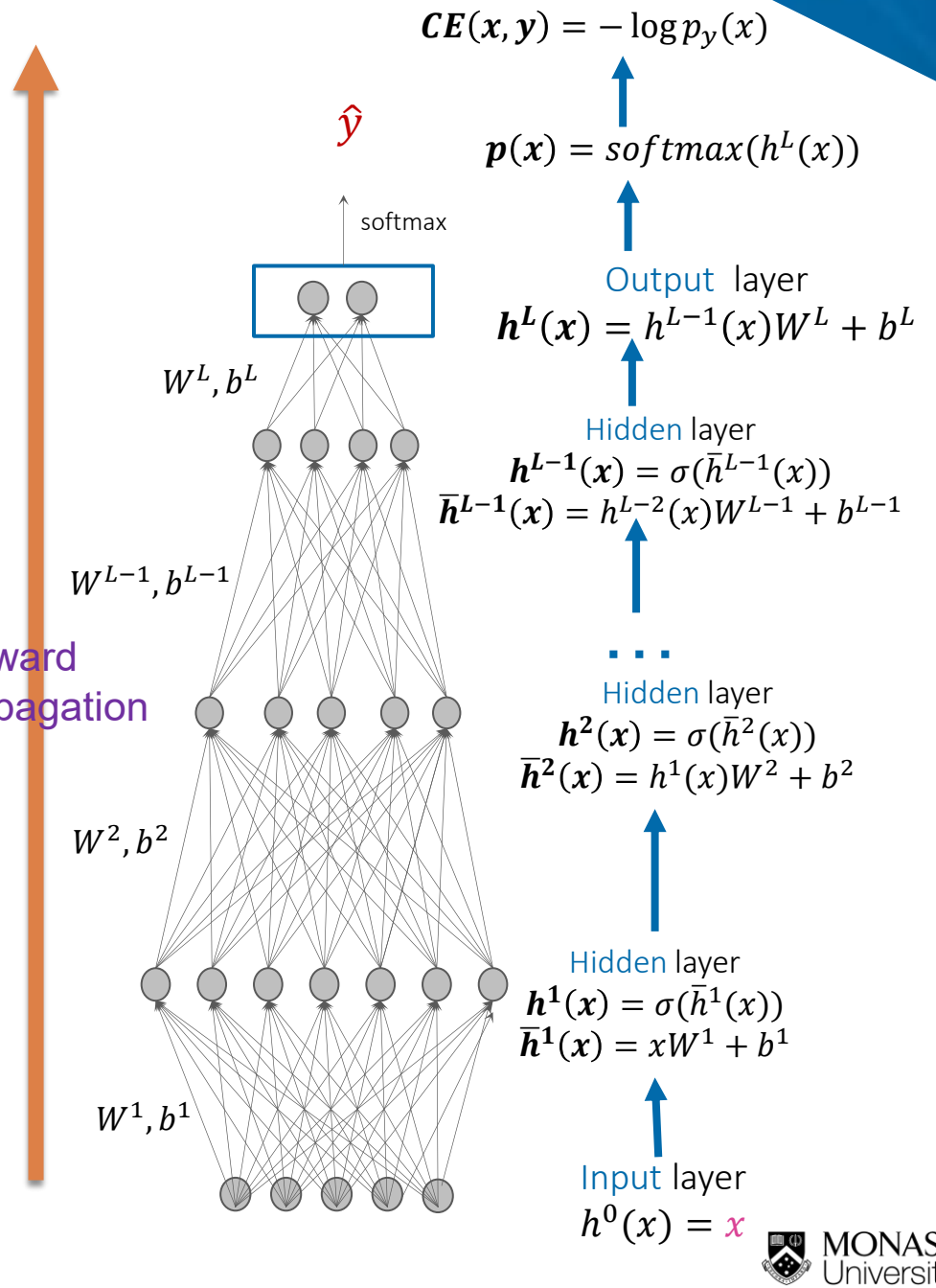
- Layer k has n_k neurons
  - $n_0 = d$ and $n_L = M$
- Weight matrices and biases
  - $W^k \in \mathbb{R}^{n_{k-1}\times n_k}$ and $b^k \in \mathbb{R}^{1\times n_k}$ for $k = 1,2,\dots,L$

DNNs: making prediction via FP

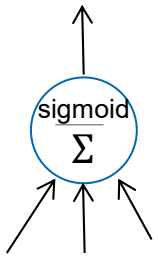
Forward propagation- Classification

$h^0(x) = x$ with the ground-truth label y
for $k = 1$ to $L-1$ do
  $\bar{h}^k(x) = h^{k-1}(x)W^k + b^k$ //linear operation
  $h^k(x) = \sigma(\bar{h}^k(x))$ //activation to bring non-linearity
$h^L(x) = h^{L-1}(x)W^L + b^L$
$p(x) = softmax(h^L(x))$ //prediction probabilities

- Parameter: $\theta = (W^k, b^k)_{k=1}^L$ is given
- $\sigma: \mathbb{R} \to \mathbb{R}$ is the **activation function**
  - Sigmoid, tanh, ReLU, and etc.
- Output: $p(x) = \text{softmax}(h^L(x))$
  - $\Pr(y = k|x) = p_k(x) = \frac{\exp\{h_k^L(x)\}}{\sum_{j=1}^M \exp\{h_j^L(x)\}}$
  - $\hat{y} = \underset{1 \le k \le M}{\text{argmax}}\, p_k(x)$

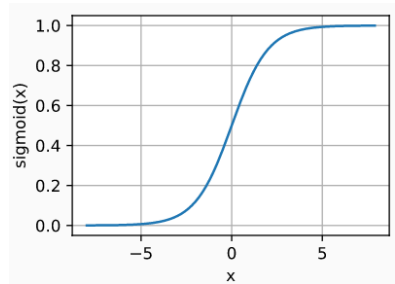


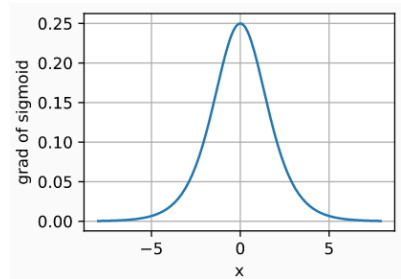
Modern Activation Functions

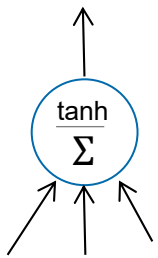


- The ***sigmoid*** function
  - $s(z) = \frac{1}{1+e^{-z}}$
  - Logistic S-shaped, continuous, and differentiable
  - Output range: 0 to 1

$$\sigma(z) = s(z) = \frac{1}{1+\exp\{-z\}} \qquad \sigma'(z) = \sigma(z)(1-\sigma(z))$$

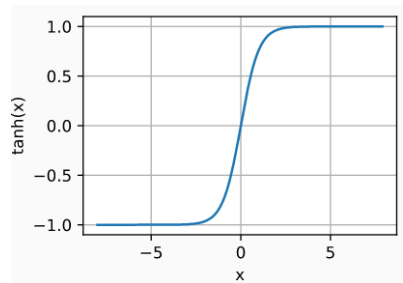


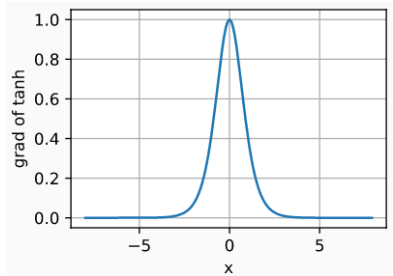


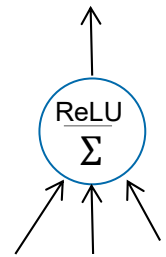


- The **hyperbolic *tangent*** function
  - $\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$
  - Logistic S-shaped, continuous, and differentiable
  - Output range: -1 to 1, hence help speed up convergence.

$$\sigma(z) = tanh(z) = \frac{\exp\{z\}-\exp\{-z\}}{\exp\{z\}+\exp\{-z\}} \qquad \sigma'(z) = 1-\sigma^2(z)$$

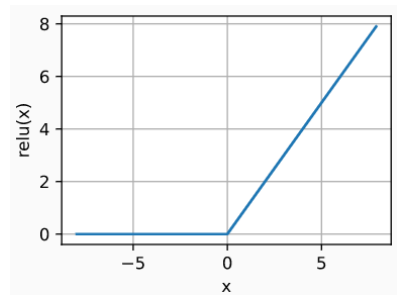


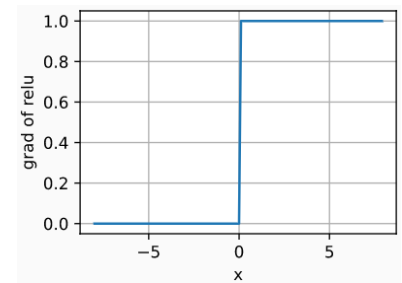




- The ***rectified linear function*** (ReLU)
  - $\mathrm{ReLU}(z) = \max(0, z)$
  - Continuous, but not differentiable at **0**
  - Hence, can make GD bounce around
  - Fast to compute, works very well in practice!
  - No cap on output value, hence help with gradient vanishing problem.

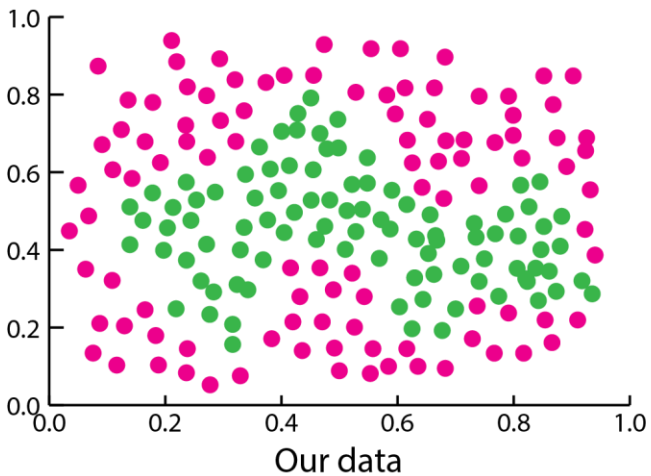
$$\sigma(z) = ReLU(z) = \max\{0, z\} \qquad \sigma'(z) = \begin{cases} 1 & if\ z \geq 0 \\ 0 & otherwise \end{cases}$$



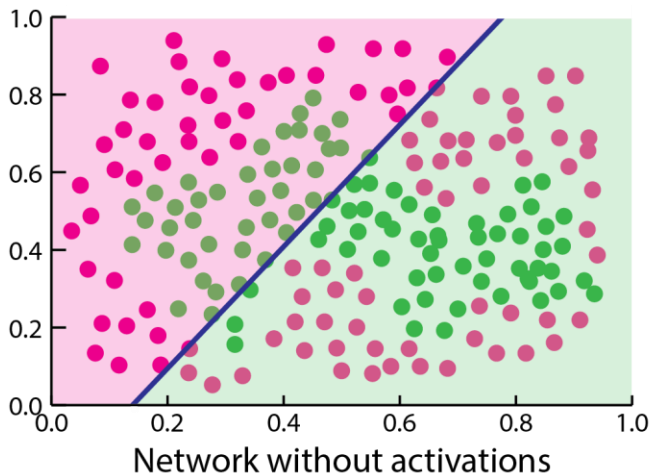


Importance of Activation Functions

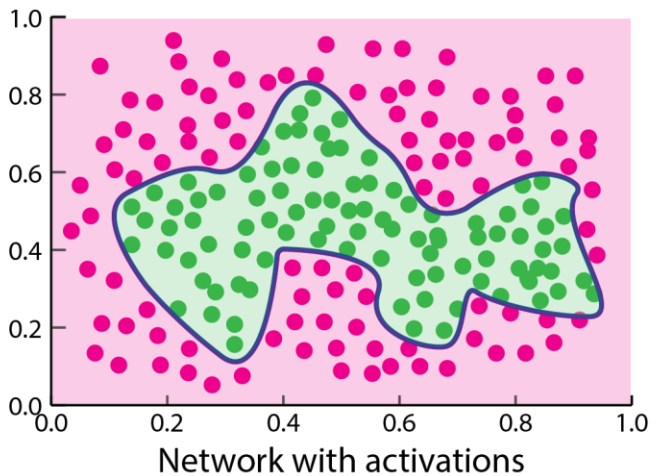
Aims to bring **non-linearities** to neural networks



Our data

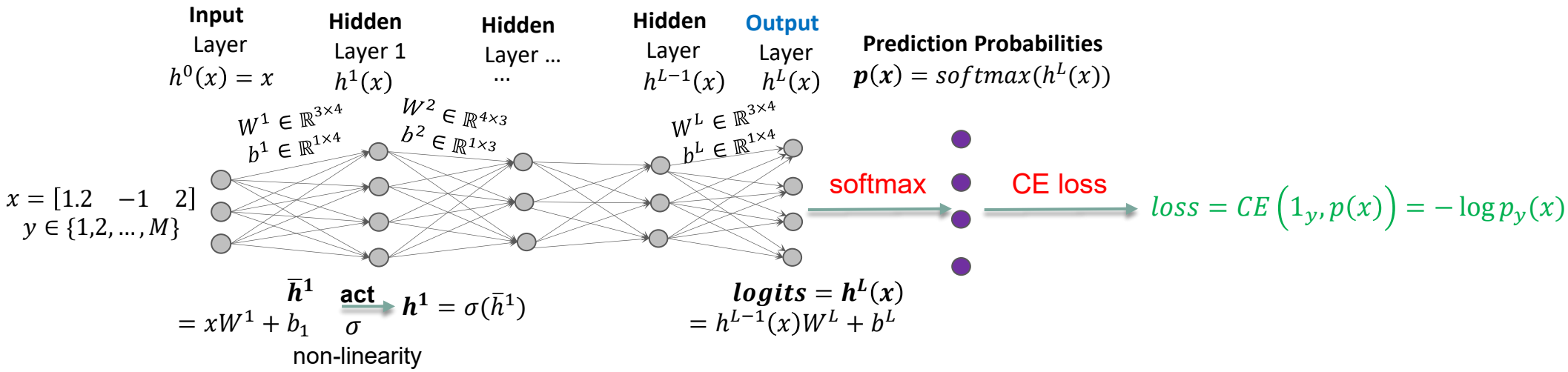


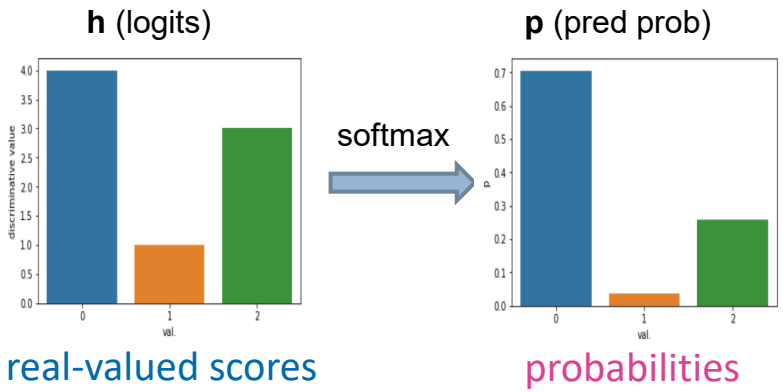
Networks without activations



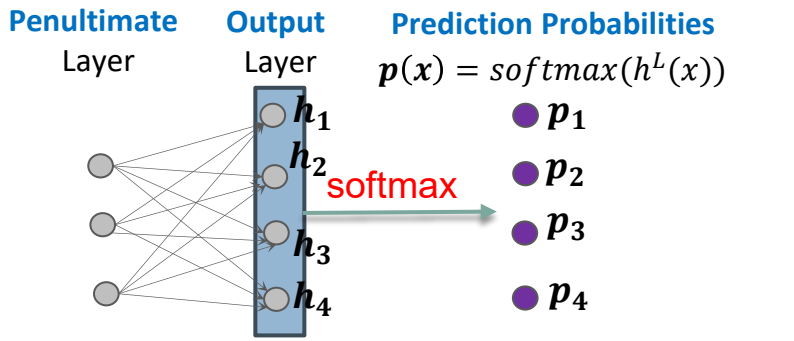
Networks with activations

Linear and Softmax Layers





transforms real-valued discriminative scores (ranged in $(-\infty, +\infty)$) to probability values (ranged in $[0, 1]$) but preserving the order.



Logit $h = [h_1\ h_2\ h_3\ h_4]$ Prob p $= [p_1\ p_2\ p_3\ p_4]$

$$p_1 = \frac{e^{h_1}}{e^{h_1} + e^{h_2} + e^{h_3} + e^{h_4}}$$

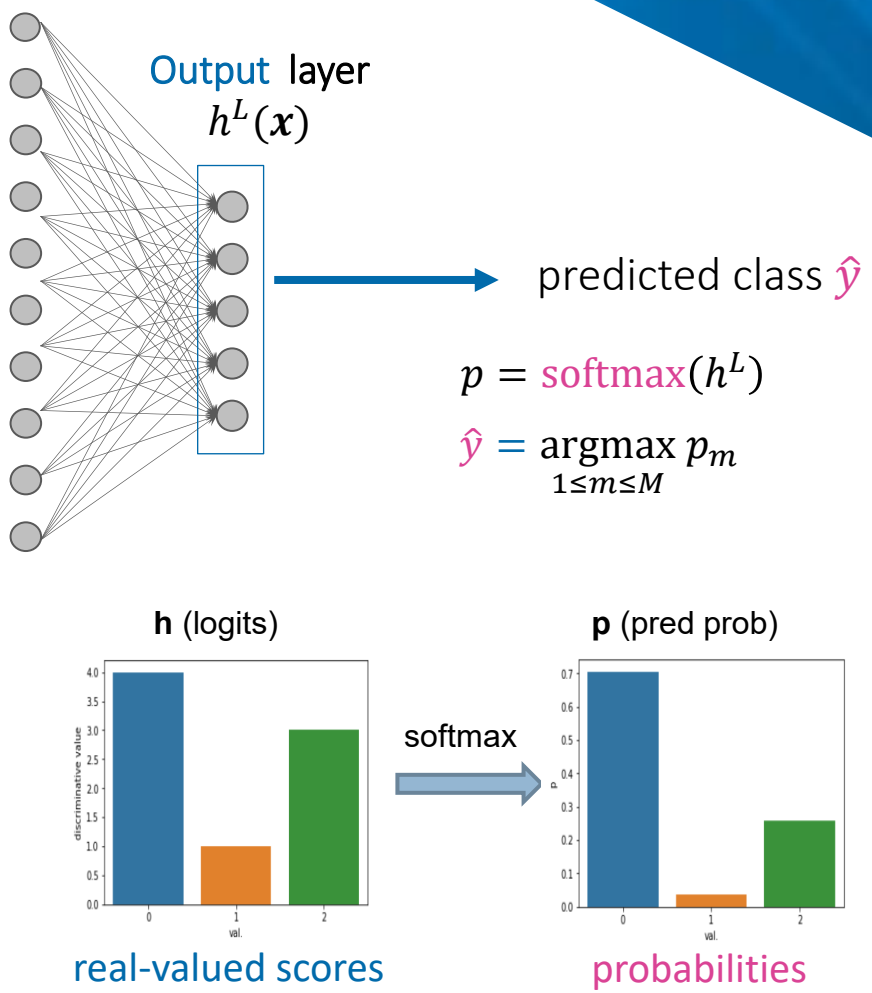
$$p_2 = \frac{e^{h_2}}{e^{h_1} + e^{h_2} + e^{h_3} + e^{h_4}}$$

$$p_3 = \frac{e^{h_3}}{e^{h_1} + e^{h_2} + e^{h_3} + e^{h_4}}$$

$$p_4 = \frac{e^{h_4}}{e^{h_1} + e^{h_2} + e^{h_3} + e^{h_4}}$$

What is a softmax layer?

- Softmax function transforms **real-valued discriminative scores h** to **discrete probabilities p**.
- Input: vector of dimension M
  - e.g., $h = [h_1, h_2, h_3] = [1, 2, 3]$, M $= 3$
- Output: a discrete distribution of dimension M
  - e.g., $p = [p_1, p_2, p_3] = [0.09, 0.24, 0.67]$
- How to calculate p_m?
  - $h = [h_m]_{m=1}^{M} \rightarrow p = \text{softmax}(h) := \left[\frac{\exp\{h_m\}}{\sum_{i=1}^{M}\exp\{h_i\}}\right]_{m=1}^{M}$
  - $p = [p_m]_{m=1}^{M}$ becomes a distribution over classes $\{1, \dots, M\}$
    - $p_m \geq 0$ $(1 \leq m \leq M)$ and $\sum_{m=1}^{M} p_m = 1$.



transforms real-valued discriminative scores (ranged in $(-\infty, +\infty)$) to probability values (ranged in $[0, 1]$) but preserving the order.

- Prediction step: return the prediction for the class that has the highest probability

$$\hat{y} = \underset{1\leq m\leq M}{\text{argmax}}\, p_m$$

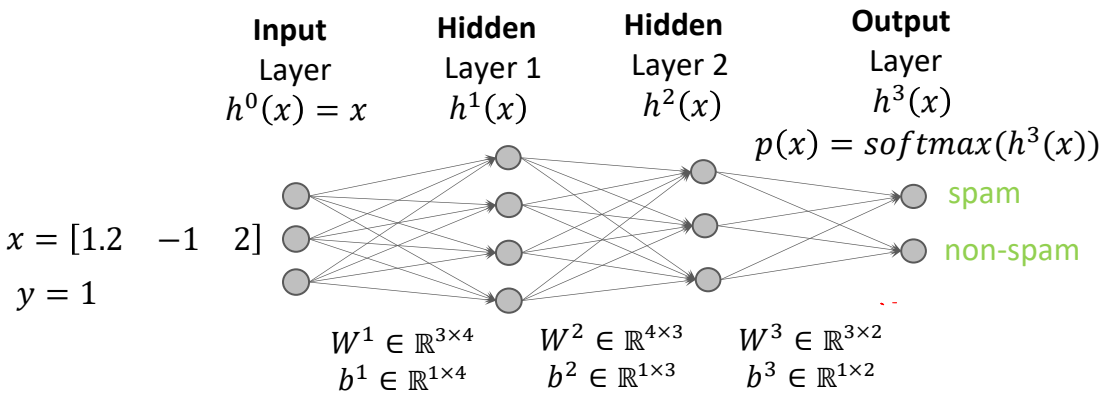
  - i.e., return 3 in the above example (why?)

DNNs: making prediction via FP

Example of spam email detection

- From emails, assume that we extract **three features**, hence data points $x \in \mathbb{R}^3$
  - $x = [x_1, x_2, x_3] \in \mathbb{R}^3$
  - There are **two** classes and labels: **spam** $(y = 1)$ and **non-spam** $(y = 2)$

- Network architecture:
  - $3 \to 4(sigmoid) \to 3(sigmoid) \to 2(softmax)$
- $h^0(x) = x = [1.2, -1, 2.2] \in \mathbb{R}^{1\times 3}$
- Hidden layer 1
  - $\bar{h}^1(x) = h^0(x)W^1 + b^1 \in \mathbb{R}^{1\times 4}$ *(linear operation)*
  - $h^1(x) = sigmoid\left(\bar{h}^1(x)\right) = \left[sigmoid\left(\bar{h}_i^1(x)\right)\right]_{i=1}^{4} \in \mathbb{R}^{1\times 4}$ *(activation)*
- Hidden layer 2
  - $\bar{h}^2(x) = h^1(x)W^2 + b^2 \in \mathbb{R}^{1\times 3}$ *(linear operation)*
  - $h^2(x) = sigmoid\left(\bar{h}^2(x)\right) \in \mathbb{R}^{1\times 3}$ *(element-wise activation)*
- Output layer
  - $h^3(x) = h^2(x)W^3 + b^3 \in \mathbb{R}^{1\times 2}$ *(linear operation)*
  - $p(x) = softmax\left(h^3(x)\right) \in \mathbb{R}^{1\times 2}$ *(softmax activation)*
- Assume that $p(x) = [0.3, 0.7]$
  - Prediction $\hat{y} = 2 \neq y = 1 \to$ ***incorrect*** *prediction*



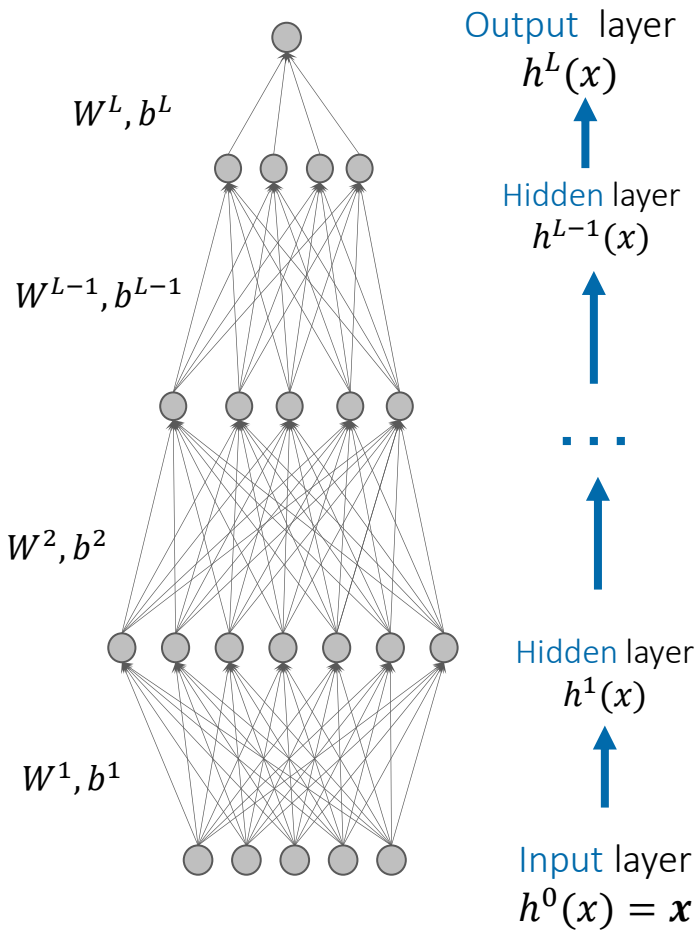
$h^0(x) = x$
for $k = 1$ to 2 do
  $\bar{h}^k = h^{k-1}(x)W^k + b^k$ //linear operation
  $h^k(x) = \sigma\left(\bar{h}^k(x)\right)$ //activation
$h^3(x) = h^2(x)W^3 + b^2$
$p(x) = softmax(h^3(x))$ //prediction probabilities

Exercise: *check all the dimensions for matrix multiplication consistency! What is the cross-entropy loss in this case?*

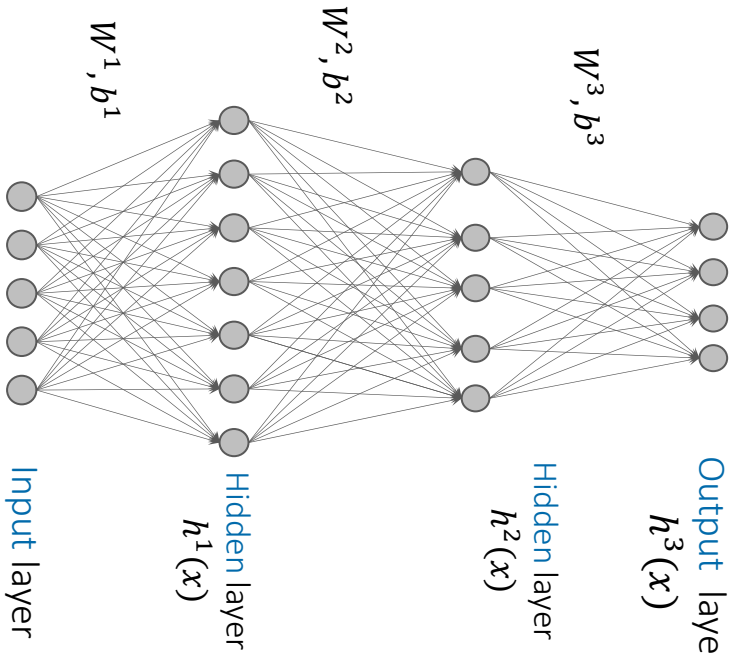
DNNs: making prediction via FP

Forward propagation- Regression

$h^0(x) = x$
for $k = 1$ to $L - 1$ do
  $\bar{h}^k(x) = h^{k-1}(x)W^k + b^k$ //linear operation
  $h^k(x) = \sigma(\bar{h}^k(x))$ //activation
$h^L(x) = h^{L-1}(x)W^L + b^L$



- Parameter: $\theta = \left(W^k, b^k\right)_{k=1}^{L}$
- $\sigma: \mathbb{R} \to \mathbb{R}$ is the activation function
  - Sigmoid, tanh, ReLU
- Output:
  - $\hat{y} = h^L(x)$



Declare FFN

```
#set the random seed for Torch
torch.manual_seed(1234)

W1 = torch.randn(5, 7)
b1 = torch.randn(1,7)
W2 = torch.randn(7, 5)
b2 = torch.randn(1,5)
W3 = torch.randn(5, 4)
b3 = torch.randn(1,4)
```

Declare one data example

```
x= torch.tensor([[1.0, 2.0, -2.0, 0.2,5.7]])
print(f'x shape: {x.shape}')
```

```
x shape: torch.Size([1, 5])
```

Forward propagation

```
hbar_1 = torch.matmul(x, W1) + b1
h1 = torch.relu(hbar_1)
print(f'hbar_1: {hbar_1}')
print(f'h1: {h1}')
```

```
hbar_1: tensor([[ 8.9391, -4.6345,  7.4453, -0.9053, -3.9509,  1.2647, -8.7399]])
h1: tensor([[8.9391, 0.0000, 7.4453, 0.0000, 0.0000, 1.2647, 0.0000]])
```

```
hbar_2 = torch.matmul(h1, W2) + b2
h2 = torch.relu(hbar_2)
print(f'hbar_2: {hbar_2}')
print(f'h2: {h2}')
```

```
hbar_2: tensor([[ -1.8582,  -3.6770,  -3.9167,   4.2753, -10.4959]])
h2: tensor([[0.0000, 0.0000, 0.0000, 4.2753, 0.0000]])
```

```
h3 = torch.matmul(h2, W3) + b3
print(f'logit h3: {h3}')
```

```
logit h3: tensor([[  3.5699, -10.3066,  -5.5052,  -2.0499]])
```

Making prediction

```
p = torch.softmax(h3, dim=1)
print(f'p: {p}')
```

```
p: tensor([[9.9627e-01, 9.3731e-07, 1.1406e-04, 3.6119e-03]])
```

```
yhat = torch.argmax(p, dim=1)
print(f'yhat: {yhat.item()}')
```

```
yhat: 0
```

```
#Assume that ground-truth label y=1 (index starts from 0)
loss = torch.mean(-torch.log(p[0,1]))
print(f'loss: {loss}')
```

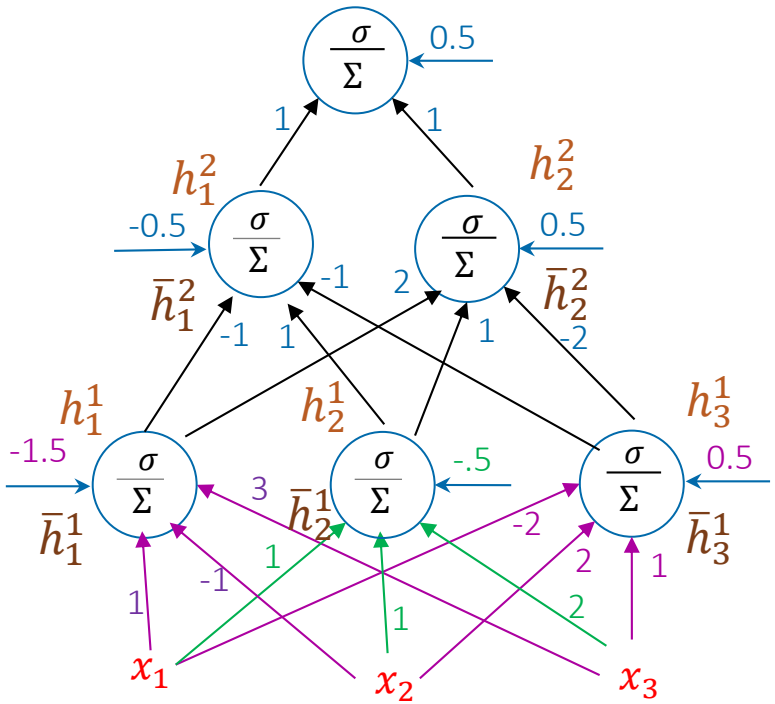
```
loss: 3.8615705966949463
```

Layer View versus Neural View

Neuron and Layer views

- Layer (macro) view

- Neuron (micro) view



Neuron and Layer views

- Layer (macro) view

$$W^1 = \begin{bmatrix} 1 & 1 & -2 \\ -1 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix}, b^1 = [-1.5 \ -0.5 \ 0.5]$$

$$\bar{\boldsymbol{h}}^{\mathbf{1}} = [\bar{h}_1^1 \quad \bar{h}_2^1 \quad \bar{h}_3^1]$$

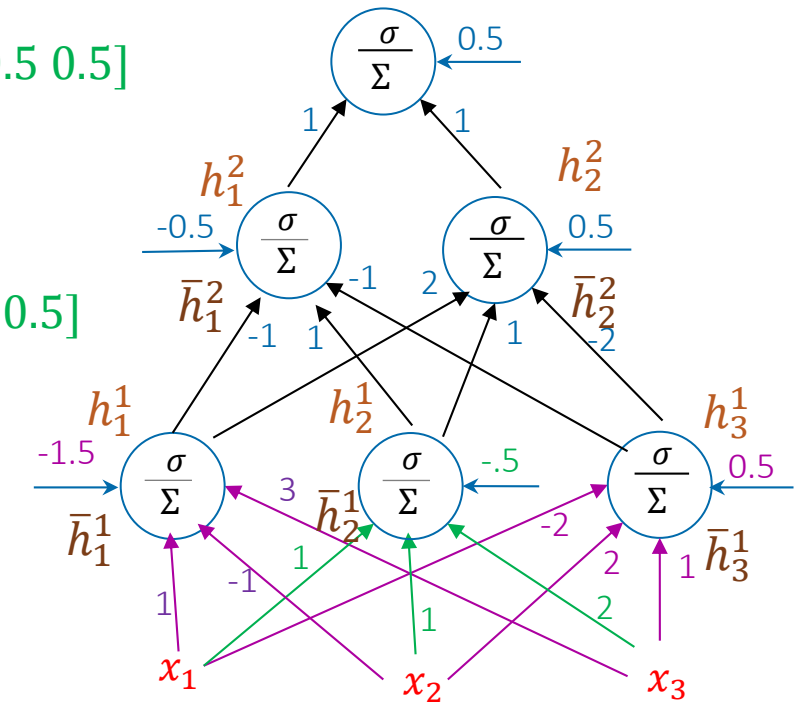
$$= [x_1 \ x_2 \ x_3] \begin{bmatrix} 1 & 1 & -2 \\ -1 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix} + [-1.5 \ -0{,}5 \ 0.5]$$

$$= W^1 x + b^1$$

$$\boldsymbol{h}^{\mathbf{1}} = \sigma(\bar{h}^1)$$

$$W^2 = \begin{bmatrix} * & * \\ * & * \\ * & * \end{bmatrix}, b^2 = [**]$$

- Neuron (micro) view



Neuron and Layer views

- Layer (macro) view

$$W^1 = \begin{bmatrix} 1 & 1 & -2 \\ -1 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix}, b^1 = [-1.5 \ -0.5 \ 0.5]$$

$$\bar{\boldsymbol{h}}^{\boldsymbol{1}} = [\bar{h}_1^1 \quad \bar{h}_2^1 \quad \bar{h}_3^1]$$

$$= [x_1 \ x_2 \ x_3] \begin{bmatrix} 1 & 1 & -2 \\ -1 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix} + [-1.5 \ -0{,}5 \ 0.5]$$

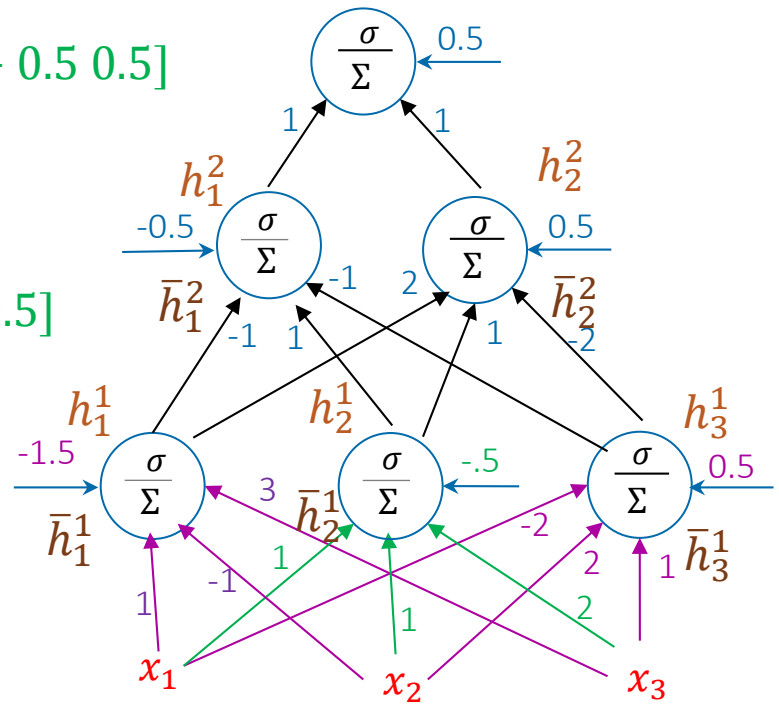
$$= W^1 x + b^1$$

$$\boldsymbol{h}^{\boldsymbol{1}} = \sigma(\bar{h}^1)$$

$$W^2 = \begin{bmatrix} -1 & 2 \\ 1 & 1 \\ -1 & -2 \end{bmatrix}, b^2 = [-0.5 \ 0.5]$$

$$\bar{\boldsymbol{h}}^{\boldsymbol{2}} = [\bar{h}_1^2 \quad \bar{h}_2^2] = h^1 W^2 + b^2$$

- Neuron (micro) view



Neuron and Layer views

Layer (macro) view

$$W^1 = \begin{bmatrix} 1 & 1 & -2 \\ -1 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix}, b^1 = [-1.5 \ -0.5 \ 0.5]$$

$$\bar{\boldsymbol{h}}^{\boldsymbol{1}} = [\bar{h}_1^1 \quad \bar{h}_2^1 \quad \bar{h}_3^1]$$

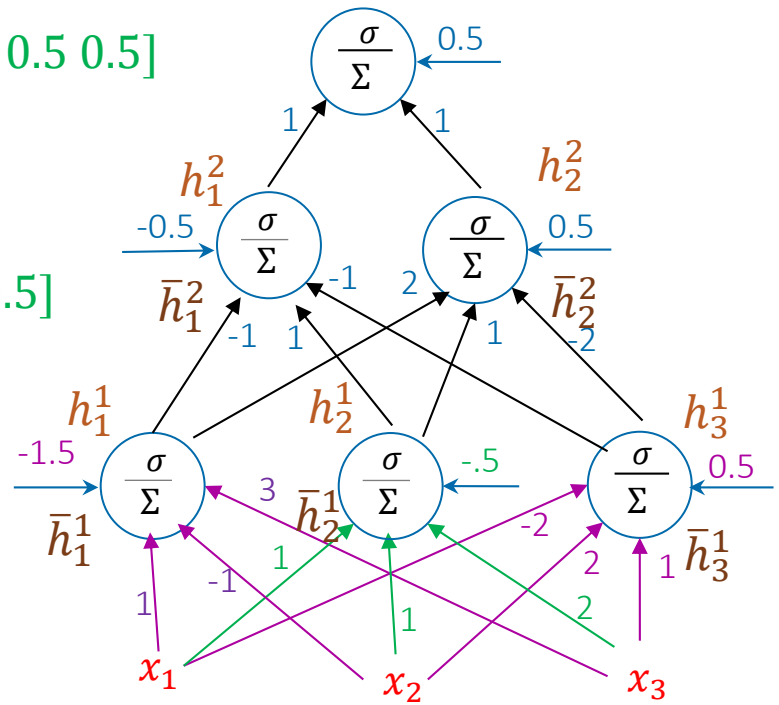
$$= [x_1 \ x_2 \ x_3] \begin{bmatrix} 1 & 1 & -2 \\ -1 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix} + [-1.5 \ -0{,}5 \ 0.5]$$

$$= W^1 x + b^1$$

$$\boldsymbol{h}^{\boldsymbol{1}} = \sigma(\bar{h}^1)$$

$$W^2 = \begin{bmatrix} -1 & 2 \\ 1 & 1 \\ -1 & -2 \end{bmatrix}, b^2 = [-0.5 \ 0.5]$$

$$\bar{\boldsymbol{h}}^{\boldsymbol{2}} = [\bar{h}_1^2 \quad \bar{h}_2^2] = h^1 W^2 + b^2$$



Neuron (micro) view

$$x = [x_1 \quad x_2 \quad x_3]$$

$$W_1^1 = \begin{bmatrix} 1 \\ -1 \\ 3 \end{bmatrix}, b_1^1 = -1.5$$

$$\bar{\boldsymbol{h}}_{\boldsymbol{1}}^{\boldsymbol{1}} = xW_1^1 + b_1^1$$

$$\boldsymbol{h}_{\boldsymbol{1}}^{\boldsymbol{1}} = \sigma(\bar{h}_1^1)$$

$$W_2^1 = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} \ b_2^1 = -0.5$$

$$\bar{\boldsymbol{h}}_{\boldsymbol{2}}^{\boldsymbol{1}} = xW_2^1 + b_2^1$$

$$\boldsymbol{h}_{\boldsymbol{2}}^{\boldsymbol{1}} = \sigma(\bar{h}_2^1)$$

$$W_3^1 = \begin{bmatrix} -2 \\ 2 \\ 1 \end{bmatrix} \ b_3^1 = 0.5$$

$$\bar{\boldsymbol{h}}_{\boldsymbol{3}}^{\boldsymbol{1}} = xW_3^1 + b_3^1$$

$$\boldsymbol{h}_{\boldsymbol{3}}^{\boldsymbol{1}} = \sigma(\bar{h}_3^1)$$



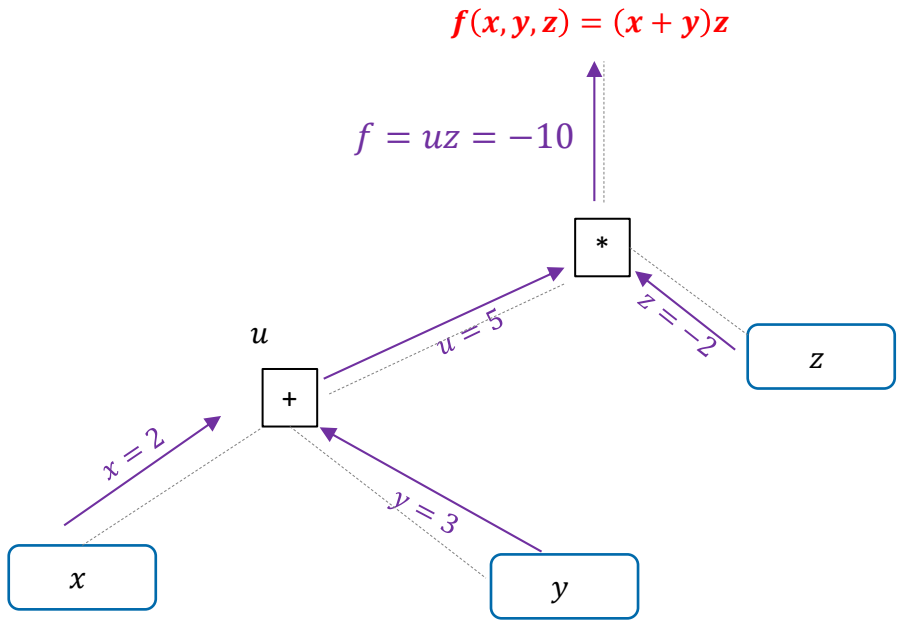
Computational graph

Computational graph (used in PyTorch)

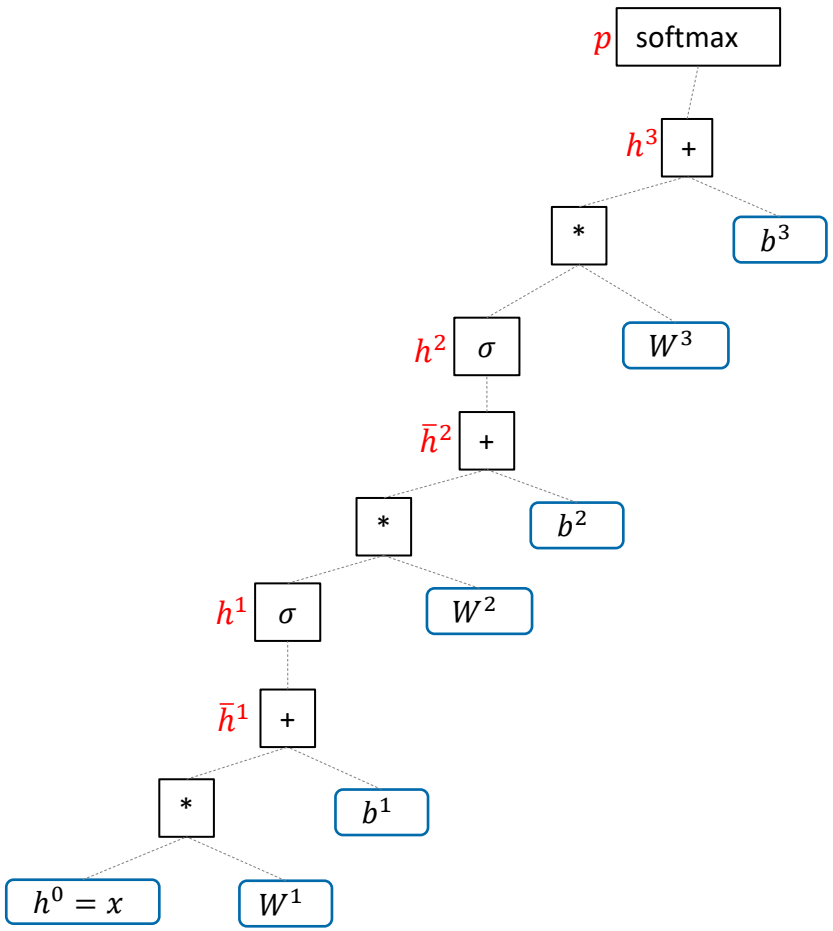
Problem: $f(x, y, z) = (x + y)z$
What is the output evaluated at $x = 2, y = 3, z = -2$?

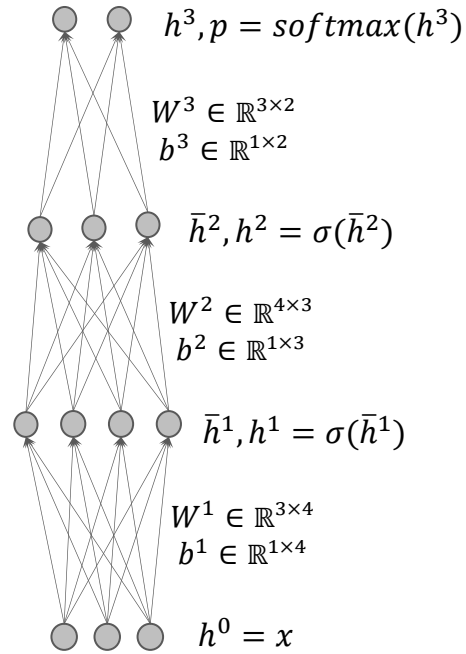
Step 1:

a) construct computational graph
b) forward propagation
c) record value at each node



Computational graph of feedforward nets





$h^0(x) = x$
for $k = 1$ to 2 do
  $\bar{h}^k = h^{k-1}(x)W^k + b^k$ //linear operation
  $h^k(x) = \sigma(\bar{h}^k(x))$ //activation
$h^3(x) = h^2(x)W^3 + b^2$
$p(x) = softmax(h^3(x))$ //prediction probabilities

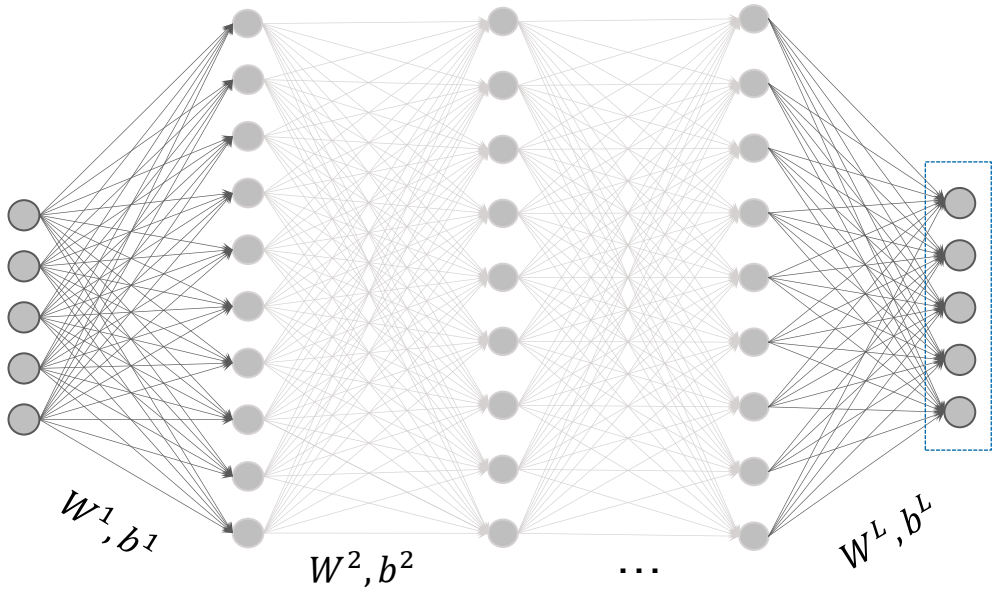

Q&A
WHO?
WHERE?
WHEN?
WHAT?
How?
Why?
When?
Where?
HOW?

Training deep neural networks

Training a deep NN

- We have learned how to make prediction if the model had already trained.
- Machine learning is all about how to train these models from data!
  - Training set $D = \{(x_1, y_1), \ldots, (x_N, y_N)\}$
- Back Propagation is currently the best approach to train a DNN.

Training deep nets



suffer loss $l(y, \hat{y}) = \mathrm{CE}\left(1_y, p(x)\right) = -\log p_y(x)$

Training set

$D = \{(x_1, y_1), \ldots (x_N, y_N)\}$

Loss function

$$L(D;\theta) \coloneqq \frac{1}{N}\sum_{i=1}^{N} CE\left(1_{y_i}, p(x_i)\right) = -\frac{1}{N}\sum_{i=1}^{N} \log p_{y_i}(x_i)$$

(negative log likelihood)

- DNN parameters: $\theta \coloneqq \{(W^l, b^l)\}_{l=1}^{L}$
- Find model parameters (weight matrices and biases) so that the model predictions fit the training set as much as possible:

$$\min_{\theta} L(D;\theta)$$

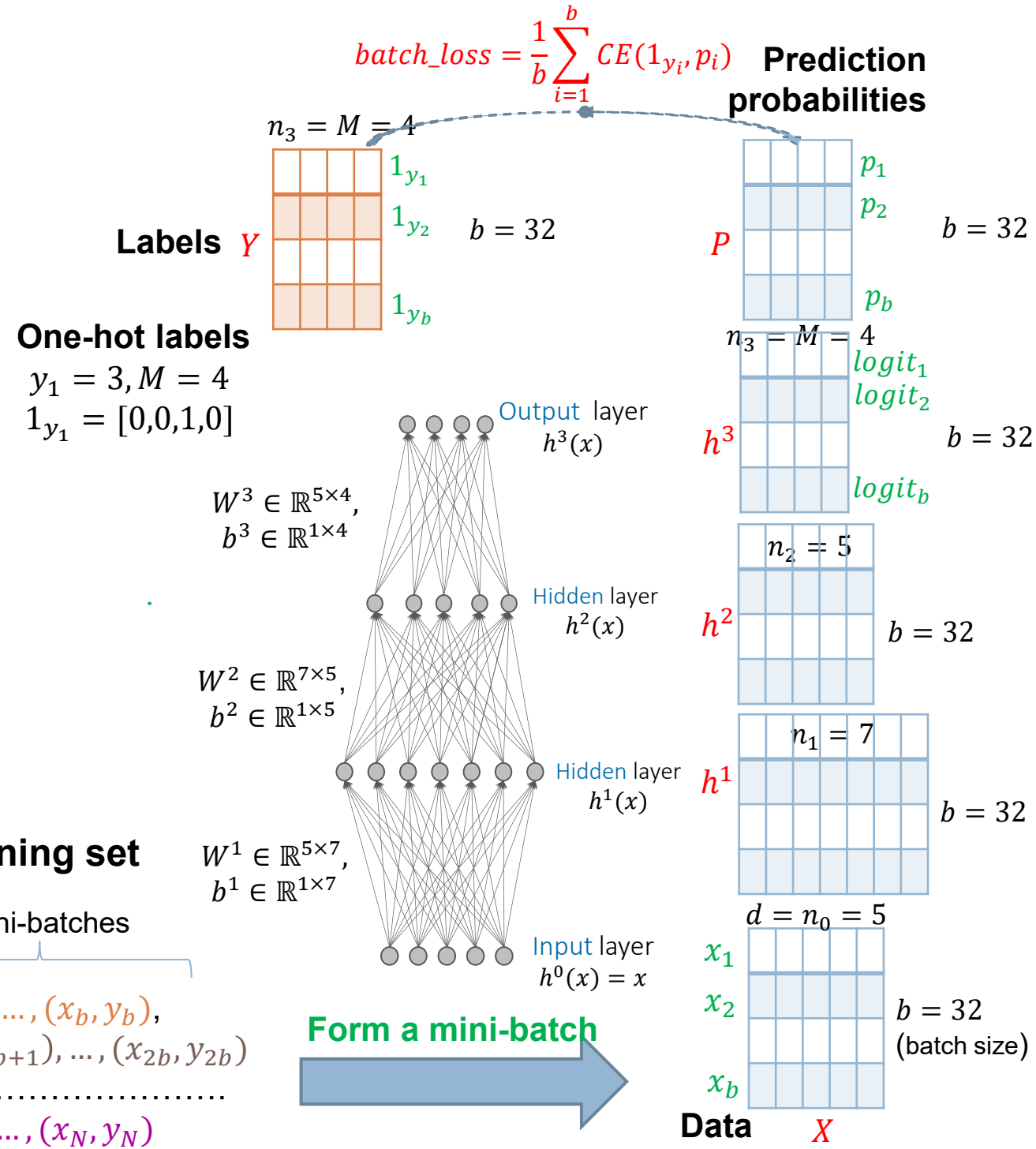
- Use optimizers SGD, Adagrad, Adam, RMSProp to update the model parameters (lectures 4)

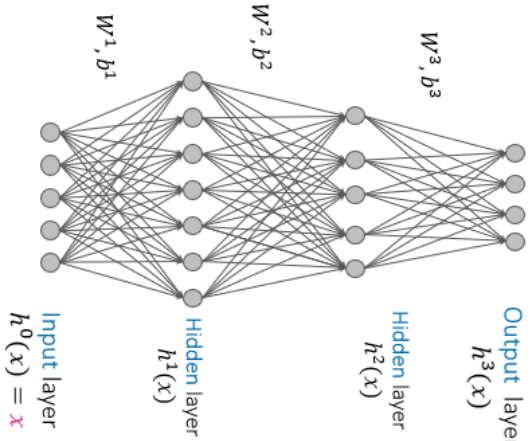
TorchOpt

```
"Adam":torch.optim.Adam
"Adadelta":torch.optim.Adadelta
"Adagrad":torch.optim.Adagrad
"Adamax":torch.optim.Adamax
"AdamW": torch.optim.AdamW
"ASGD":torhc.optim.ASGD,
"NAdam":torch.optim.NAdam,
"RMSprop":torhc.optim.RMSprop
"RAdam":torch.optim.RAdam
"Rprop": torch.optim.Rprop
"SGD":torch.optim.SGD
```

Mini-batch feed-forward

- **Input**
  - Batch X: $[b, n_0 = d = 5]$ (b is the batch size)
- **Hidden layer 1**
  - $\boldsymbol{h^1 = \sigma(XW^1 + b^1)}$
  - $Tensor\,[b, n_1 = 7]$
- **Hidden layer 2**
  - $\boldsymbol{h^2 = \sigma(h^1W^2 + b^2)}$
  - Tensor $[b, n_2 = 7]$
- **Output layer**
  - $\boldsymbol{h^3 = h^2W^3 + b^3}$
  - $P = softmax(h^3, dim = 1)$
  - $Tensor\,\mathrm{P}$: $[b, n_L = M = 4]$
- **The loss of the batch**
  - $\frac{1}{b}\sum_{i=1}^{b} CE(1_{y_i}, p_i) = -\frac{1}{b}\sum_{i=1}^{b} \log p^i_{y_i}$
  - Update weight matrices and biases to **minimize** the batch loss.





Declare FFN

```
#set the random seed for Torch
torch.manual_seed(1234)

W1 = torch.randn(5, 7)
b1 = torch.randn(1,7)
W2 = torch.randn(7, 5)
b2 = torch.randn(1,5)
W3 = torch.randn(5, 4)
b3 = torch.randn(1,4)
```

Declare a batch

```
x= torch.rand(size=(4,5))
print(f'x: {x}')

x: tensor([[0.3574, 0.1191, 0.9922, 0.8757, 0.7378],
        [0.9949, 0.2338, 0.2153, 0.2073, 0.4758],
        [0.0586, 0.8958, 0.5129, 0.7490, 0.2254],
        [0.4485, 0.5658, 0.3631, 0.9719, 0.2716]])
```

Forward propagation

```
hbar_1 = torch.matmul(x, W1) + b1
h1 = torch.relu(hbar_1)
print(f'hbar_1.shape: {hbar_1.shape}')
print(f'h1.shape: {h1.shape}')

hbar_1.shape: torch.Size([4, 7])
h1.shape: torch.Size([4, 7])
```

```
hbar_2 = torch.matmul(h1, W2) + b2
h2 = torch.relu(hbar_2)
print(f'hbar_2.shape: {hbar_2.shape}')
print(f'h2.shape: {h2.shape}')

hbar_2.shape: torch.Size([4, 5])
h2.shape: torch.Size([4, 5])
```

```
h3 = torch.matmul(h2, W3) + b3
print(f'logit h3: {h3}')

logit h3: tensor([[ 0.4458, -3.1415, -2.9431, -0.9237],
        [ 0.1560, -2.4655, -2.6027, -0.4142],
        [ 1.0816, -1.8671, -1.6175, -0.7504],
        [ 0.8463, -1.9495, -1.8475, -0.7677]])
```

```
p = torch.softmax(h3, dim=1)
print(f'p: {p}')

p: tensor([[0.7601, 0.0210, 0.0257, 0.1932],
        [0.5877, 0.0427, 0.0372, 0.3323],
        [0.7814, 0.0410, 0.0526, 0.1251],
        [0.7531, 0.0460, 0.0509, 0.1499]])
```

Making prediction

```
yhat = torch.argmax(p, dim=1)
print(f'yhat: {yhat}')

yhat: tensor([0, 0, 0, 0])
```

```
#Assume that ground-truth label y = [0,1,0,1]
y = torch.tensor([0,1,0,1])
print(f'y: {y}')
one_hot_y = torch.nn.functional.one_hot(y,num_classes=4)
print(f'one_hot_y: {one_hot_y}')
print(one_hot_y * torch.log(p))
loss = -torch.mean(one_hot_y * torch.log(p))
print(f'loss: {loss}')

y: tensor([0, 1, 0, 1])
one_hot_y: tensor([[1, 0, 0, 0],
        [0, 1, 0, 0],
        [1, 0, 0, 0],
        [0, 1, 0, 0]])
tensor([[-0.2743, -0.0000, -0.0000, -0.0000],
        [-0.0000, -3.1530, -0.0000, -0.0000],
        [-0.2467, -0.0000, -0.0000, -0.0000],
        [-0.0000, -3.0793, -0.0000, -0.0000]], grad_fn=<MulBackward0>)
loss: 0.42208024859428406
```

Training FFNs using Mini-batches

Training set

1 epoch

$y_1 = cat$ $y_2 = dog$ $y_{64} = lion$ $y_* = spider$ $y_{*+63} = cow$ $y_{60000} = cat$

x_1 x_2 x_{32} x_* x_{*+31} x_{60000}

A mini-batch

One vector: [1, 5]
→ Batch: [32, 5]

Another mini-batch

Our FFN

32

5

Input Layer $h^0(x) = x$

Hidden Layer 1 $h^1(x)$

Hidden Layer 2 $h^2(x)$

Output Layer $h^3(x)$

$p(x) = softmax(h^3(x))$

$W^1 \in \mathbb{R}^{3\times4}$ $b^1 \in \mathbb{R}^{4\times1}$

$W^2 \in \mathbb{R}^{4\times3}$ $b^2 \in \mathbb{R}^{3\times1}$

$W^3 \in \mathbb{R}^{3\times2}$ $b^3 \in \mathbb{R}^{2\times1}$

$$batch_loss(\theta) = \frac{1}{32}\sum_{i=1}^{32} CE(1_{y_i}, p(x_i, \theta))$$

Update $\theta = [W^l, b^l]_{l=1}^{L}$

Optimizer
- Adam,
- RMSProp,
- SGD

$$\min_\theta batch_loss(\theta)$$

Testing set

Model parameters $\theta = [W^l, b^l]_{l=1}^{L}$

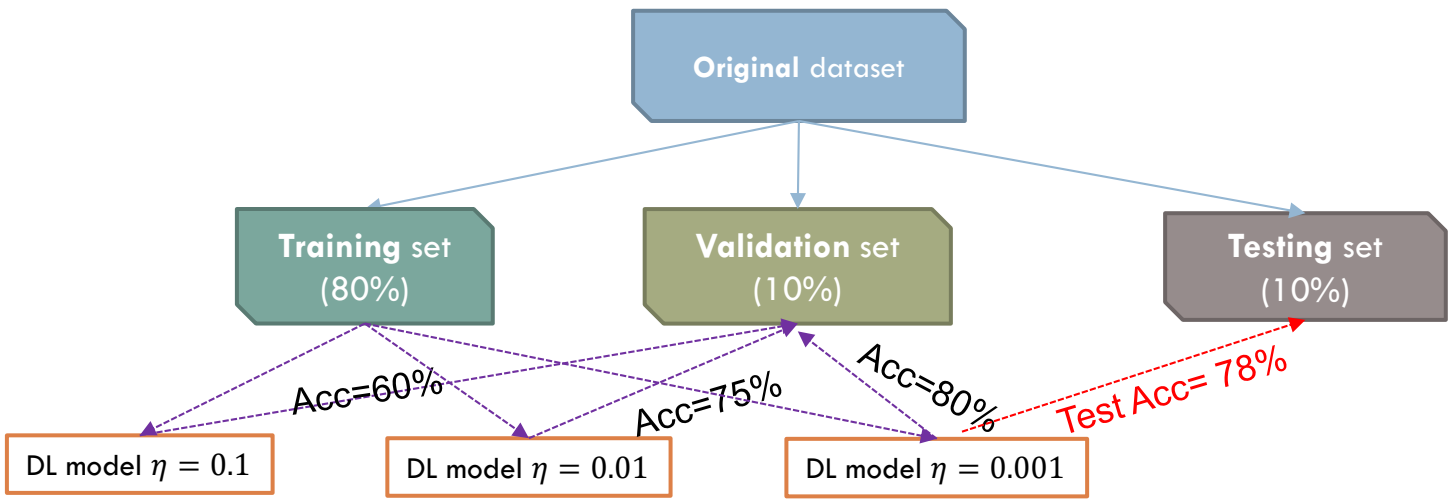
x'_1 x'_2 x'_{1000}

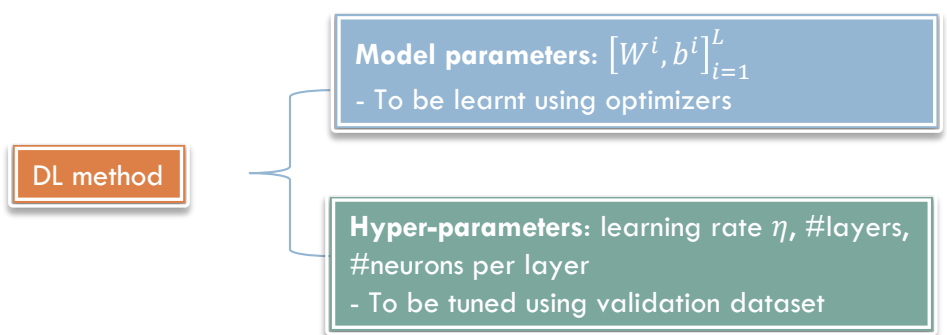
Test accuracy

Deep learning pipeline

Tuning hyper-parameters

- We want to train our DL model on a **training set** such that the **trained model** can predict well **unseen data** in a **separate testing set**.





Summary

- Prelude to deep learning
  - Feed-forward NN (vector data), Convolutional NN (image, 2D-3D grid data), Recurrent NN (sequential data).
- Perceptron
  - Stack perceptron and activation function → deep neural nets.
- Feed-forward neural networks
  - Parameterization
  - Forward propagation and compute the cross-entropy loss
  - Computational graph of deep nets
  - Train deep nets
  - Representation learning of deep nets

Thanks for your attention!

